AWS CERTIFIED GENERATIVE AI DEVELOPER, PROFESSIONAL · AIP-C01

Study treatise: one path from first principles to exam answer

Scope: the five exam domains, their task statements, the services they name, and the traps AWS reuses. Built to be read in order, top to bottom. Read each section before the next.

Sources. AWS documentation pages and the AWS exam guide for AIP-C01. The study guide repository was excluded as a knowledge source. Claims marked **Not in source** could not be confirmed on an official page in this build. Claims marked **Inference** are reasoning from the sources, not AWS statements. Figures in examples are illustrative.

Status of checks. Domains 1 to 5 were each fact-checked against AWS documentation by a review pass. Two checks were re-run on the official pages in the last pass. Items marked **Not in source** stay open. Re-check every open item before exam day.

PART 0

The exam at a glance

The exam has 75 questions. Sixty-five are scored and ten are unscored. The exam uses multiple choice and multiple response questions. The passing score is 750 on a scale from 100 to 1,000.

Item	Value
Exam name	AWS Certified Generative AI Developer, Professional
Exam code	AIP-C01
Questions	75 total. 65 scored. 10 unscored.
Question types	Multiple choice (one correct answer) and multiple response (two or more correct answers)
Passing score	750 on a scale from 100 to 1,000
Scoring by section	Not in source. The pass decision uses the whole exam.
Duration and fee	180 minutes and 300 USD. Stated in the repository README. Not on the official certification page. Confirm before exam day.

Domain weights

Domain	Name	Weight in the repository README	Approximate question count (Inference)
1	Foundation model integration, data management, and compliance	31%	about 20
2	Implementation and integration	26%	about 17
3	AI safety, security, and governance	20%	about 13
4	Operational efficiency and optimization	12%	about 8
5	Testing, validation, and troubleshooting	11%	about 7

The weights come from the repository README. The official guide page did not show them in this build. Confirm each weight against the official exam guide.

Target candidate

AWS expects two or more years of production application experience. It also expects one year of hands-on generative AI work. The exam assumes working knowledge of AWS compute, storage, networking, security, and infrastructure as code. Model training, advanced machine learning, and data engineering are outside the scope.

PART 1

How AWS asks questions

Each question has a company, a constraint, and a design that is broken or incomplete. Several options look correct. One option meets every constraint at the same time.

1. Read the last sentence of the stem. Note what it asks for.
2. List each constraint. Examples are a cost ceiling, a latency budget, a compliance rule, and an existing AWS footprint.
3. Pick the option that meets all constraints.
4. Choose the simplest managed option. Choose custom work only when a constraint forces it.

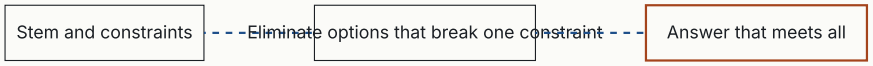


Diagram: the elimination order for every question. Built in SVG for this treatise. Inference from the exam guide.

PART 1A · COMPONENTS AND ARCHITECTURE

The components, how they connect, and where each one sits in the exam

This part introduces each AWS component before the domain detail. Read the six architectures first. Each architecture shows one request or one lifecycle. Then read the component table. Each row names the domain and task that test the component.

Diagram key. Solid boxes are components. Red outline marks a safety or control point. Animated lines show data flow. Each diagram is an SVG drawn for this treatise from the component behavior in the official pages. Official AWS images follow each group where a page provides one.

Architecture 1 · The request path

Every production call passes through an entry point, a compute step, a safety check, and a model. The request path puts those parts in order. Configuration sits beside the compute step. Telemetry sits below the path.

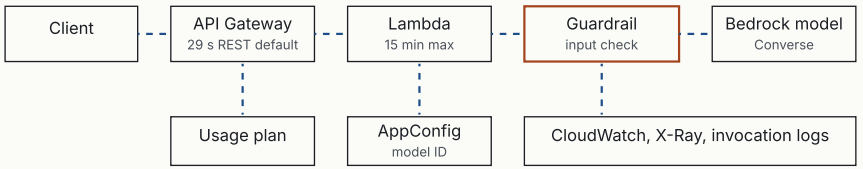


Diagram: request path. Lambda reads the model ID from configuration. The guardrail checks input before the model call. Built in SVG for this treatise. Domains 2 and 3, tasks 2.3, 2.4, 3.1.

Architecture 2 · The agent path

An agent plans steps and calls tools. Bedrock Agents and AgentCore both fit this path. The agent calls an action group or an MCP tool through a gateway. Policy and identity sit in front of each tool call. Trace records each step.

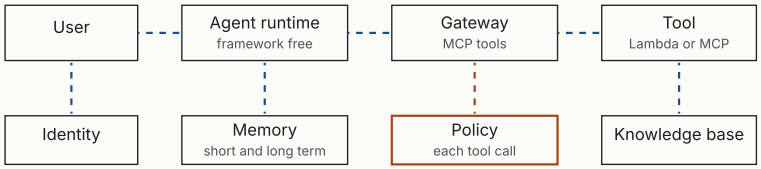


Diagram: agent path. Policy checks each tool call before it runs. Memory holds context between turns. Built in SVG for this treatise. Domains 2 and 3, tasks 2.1, 2.2, 3.1.

Architecture 3 · The safety and privacy path

Safety has layers. Each layer uses a different control. A gap in one layer is covered by another. Data controls sit below the path. They protect data at rest and in transit, and they restrict who reads it.

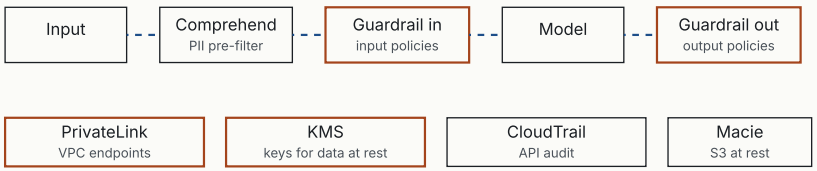


Diagram: safety layers on the request path and controls below. Built in SVG for this treatise. Domain 3, tasks 3.1 and 3.2.

Architecture 4 · The knowledge path

Knowledge Bases manage the ingest path and the query path. You choose the vector store. You choose the chunking strategy when you create the data source. Retrieval returns citations. Governance keeps tenant data apart.

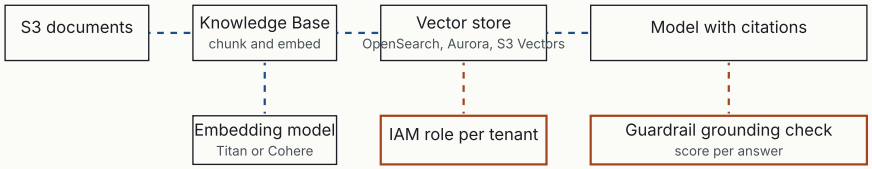


Diagram: knowledge path with the embedding step and tenant control. Built in SVG for this treatise. Domain 1, tasks 1.3 to 1.5. Domain 3, task 3.2.

Architecture 5 · The operations path

Operations answers three questions. Is the system slow? Is it spending too much? Is the answer wrong? Metrics answer the first two. Traces find the slow step. Logs hold the payloads that show the wrong answer.

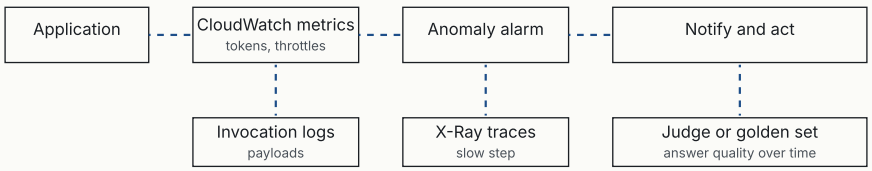


Diagram: operations signals. Metrics, traces, logs, and quality checks each answer a different question. Built in SVG for this treatise. Domains 4 and 5, tasks 4.3, 5.1, 5.2.

Architecture 6 · The release path

A change to a prompt, a model, a chunk setting, or a guardrail is a release. Each release passes the same gates. A failed gate blocks the release. A canary limits the blast radius. Rollback returns to the last approved version.

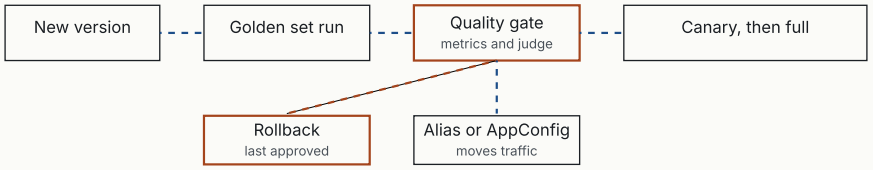


Diagram: release gates with rollback. Built in SVG for this treatise. Domain 5, tasks 5.1 and 5.2. Domain 2, task 2.3.

Component table

Each row names one component, its job, the architecture box it sits in, and the tasks that test it.

Component	Job	Sits in	Tasks
Bedrock Converse and InvokeModel	Call foundation models. Converse for chat and tools. InvokeModel for embeddings and provider bodies.	Model call	1.2, 2.4
Provisioned Throughput	Reserves model units for steady traffic. Calls must use the provisioned model ARN.	Capacity	2.2, 4.1
Cross-Region inference profile	Spreads requests across Regions in the profile for throughput and availability.	Model call	1.2, 4.1
Batch inference	Offline jobs from S3 to S3. 50 percent lower price for select models.	Offline lane	2.2, 4.1
Prompt caching	Reuses an exact prefix marked with a cache checkpoint. Stable content first.	Prompt build	2.5, 4.1
Bedrock Guardrails	Six policies on input and output. Draft, then version.	Safety boxes	3.1, 2.3
ApplyGuardrail	Checks content from any model. Does not call the model.	Safety boxes	2.3, 3.1
Knowledge Bases	Managed RAG: sync, chunk, embed, retrieve, and cite.	Knowledge path	1.4, 1.5, 5.2
Bedrock Agents	Agents with action groups, DRAFT, versions, and aliases. No longer open to new customers. AgentCore is the documented alternative.	Agent path	2.1
AgentCore Runtime	Hosts agent code from any framework. Remote MCP servers need Streamable HTTP.	Agent path	2.2
AgentCore Gateway	Exposes tools as MCP tools through one endpoint.	Agent path	1.5, 2.3
AgentCore Memory	Short-term and long-term context for an agent.	Agent path	2.1
AgentCore Identity	Controls who and what can call agents and tools.	Agent path	3.2
AgentCore Policy	Checks each tool call through Gateway before the tool runs.	Agent path	2.1, 3.1
AgentCore Observability	Traces and metrics for agent steps.	Operations path	4.3
Prompt Management	Templates, variables, versions, and variants.	Release path	1.6, 2.5

Prompt Flows	Console prompt chains with branches and Knowledge Base or Lambda nodes.	Workflow	1.6
Model evaluation with a judge	Scores responses on your dataset. Calibrate the judge first.	Release path	5.1
Bedrock Data Automation	Extracts fields from documents, images, audio, and video.	Data	1.3, 2.5
Amazon Comprehend	Detects entities and PII in single text documents up to 100 KB in real time.	Pre-filter	1.3, 3.2
Amazon Transcribe	Speech to text with speaker labels.	Data	1.3
Amazon Macie	Finds sensitive data in S3 objects at rest.	Data at rest	3.2
AWS Glue Data Quality	Rule-based quality checks written in DQDL.	Validation	1.3
AWS Lake Formation	Column and row access for lake tables.	Access	3.2, 3.3
SageMaker Model Cards	Versioned record of intended use, limits, and evaluation results.	Governance	3.3, 3.4
SageMaker Model Registry	Groups model versions and tracks approval status.	Lifecycle	1.2
SageMaker endpoints	Real-time, serverless, and asynchronous hosting.	Deploy	2.2

SageMaker Model Monitor	Drift on structured features and predictions. Not free-text quality. The docs say it is no longer open to new customers.	Operations path	4.3
OpenSearch Serverless	Managed vector search that combines filters and vectors.	Vector store	1.4
S3 Vectors	Vector storage for large, infrequent search with no cluster to run.	Vector store	1.4
Aurora with pgvector	Vector search in a relational database.	Vector store	1.4
Amazon Neptune	Graph queries for GraphRAG over explicit relationships.	Vector store	1.4
DynamoDB	Session and customer memory with TTL.	Agent path	2.1
AWS Step Functions	Durable workflows with waits, retries, and callback task tokens. Standard runs up to one year.	Orchestration	1.6, 2.2
AWS Lambda	Short compute steps. Standard functions run up to 15 minutes.	Compute	2.2
API Gateway	Entry point with throttling. REST integrations default to 29 seconds.	Request path	2.3, 3.1
AWS AppConfig	Runtime configuration, such as the model ID.	Request path	1.2
CloudWatch	Metrics, alarms, anomaly detection, and Logs Insights.	Operations path	4.3, 5.2

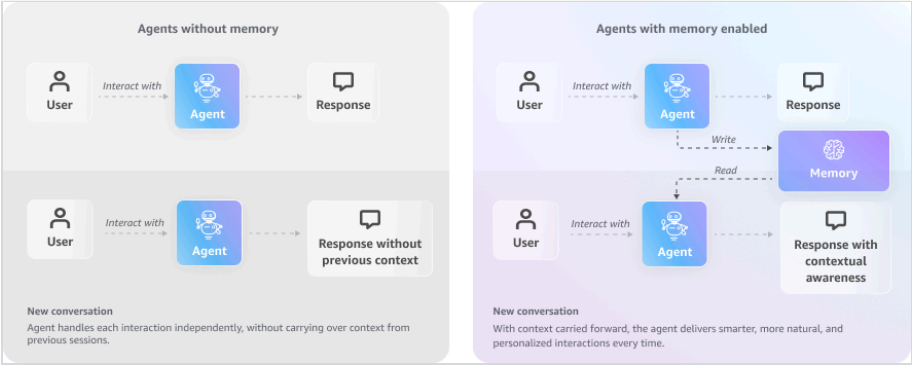
AWS X-Ray	Traces requests and downstream calls across services.	Operations path	4.3
AWS CloudTrail	Records API calls for audit. Does not show row content.	Governance	3.3
AWS KMS	Keys for encryption at rest.	Security	3.2
AWS PrivateLink	VPC interface endpoints for Bedrock planes.	Network	3.2
SCP and IAM condition keys	Enforce a rule across accounts, such as a required guardrail identifier.	Governance	3.3
AWS Outposts	AWS infrastructure on premises for data that must stay on site.	Network	2.3
Amazon Q Developer	Coding help in the IDE.	Apps	2.5
Amazon Q Business	Company chat over company data. No longer open to new customers. Amazon Quick is the documented alternative.	Apps	2.5
Amazon Kendra	Enterprise search. No longer open to new customers. Knowledge Bases is the documented alternative.	Legacy	1.3

Official images for these components

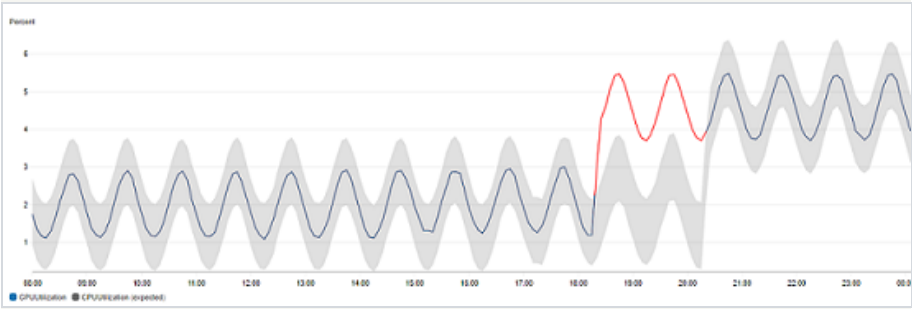
These six images come from new official pages. Earlier sections already use the other official images.



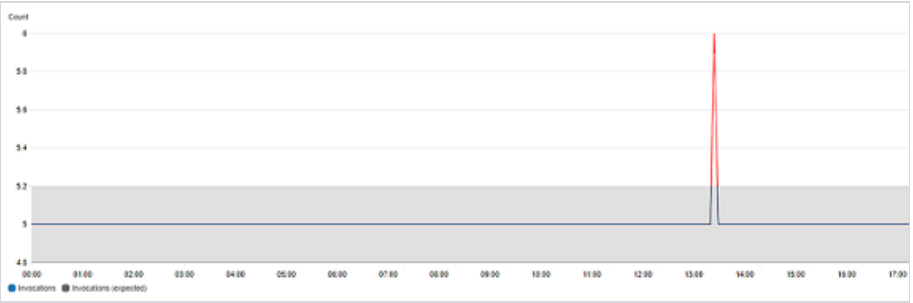
Official AWS image. Source: [Amazon API Gateway, welcome]. Use for the entry point role.



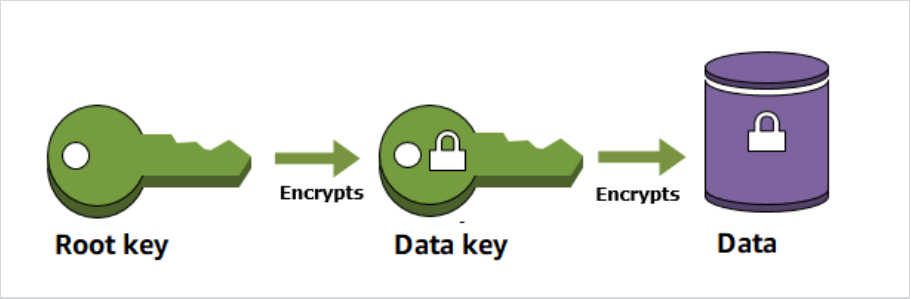
Official AWS image. Source: [AgentCore Memory]. Use for short-term and long-term memory.



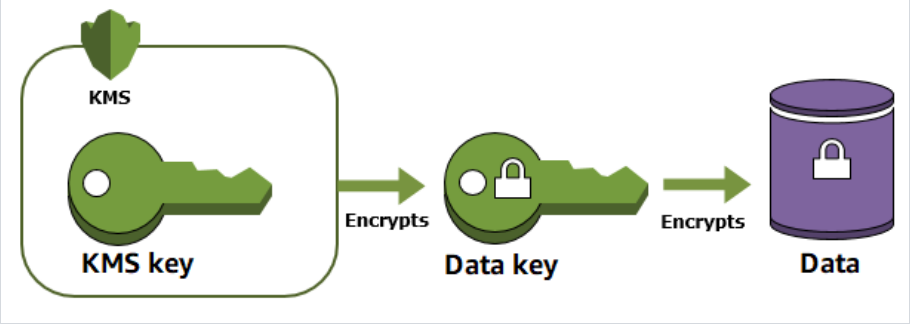
Official AWS image. Source: [CloudWatch anomaly detection]. Use for the adaptive band that alarms on token spikes.



Official AWS image. Source: CloudWatch anomaly detection.



Official AWS image. Source: AWS KMS overview. Use for the key hierarchy.



Official AWS image. Source: AWS KMS overview.

How the components tie to the domains

Domain	Main components	Architecture
1. Foundation models, data, compliance	Converse, InvokeModel, Knowledge Bases, vector stores, AppConfig, Data Automation, Comprehend, Glue Data Quality, Model Registry	Architectures 1 and 4
2. Implementation and integration	Bedrock Agents, AgentCore, Step Functions, Lambda, API Gateway, SageMaker endpoints, Batch inference	Architectures 1, 2, and 6
3. Safety, security, governance	Guardrails, Comprehend, PrivateLink, KMS, Macie, Lake Formation, CloudTrail, Model Cards, SCP	Architecture 3
4. Operational efficiency	Provisioned Throughput, Cross-Region profiles, prompt caching, Batch, CloudWatch, X-Ray, AgentCore Observability	Architecture 5
5. Testing, validation, troubleshooting	Judge evaluations, golden sets, CloudWatch Logs Insights, X-Ray, Glue Data Quality, Step Functions for release gates	Architectures 5 and 6

PART 1B · CONCEPTS COURSE

Build the vocabulary first, then the services

Each lesson starts from one first principle. The next lesson depends on the one before it. Read the lessons in order. The exam uses these ideas in every domain. If a question uses a word you cannot define, return to the lesson for that word.

Lesson 1 · Tokens and the context window

First principle. A model reads and writes tokens. A token is a piece of text, often a word part. The model has a fixed limit on the tokens it can hold in one call. That limit is the context window.

Input tokens include the instructions, the conversation, and any retrieved text. Output tokens are the reply. Both count toward the context window.

Both are billed. Output tokens usually cost more per token than input tokens. Count tokens before the call to avoid overflow. CountTokens estimates the count for a supported model.

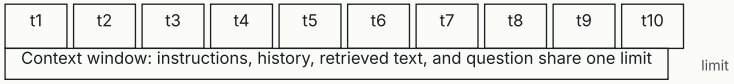


Diagram: tokens fill one context window. Built in SVG for this treatise. Schematic, not official.

Exam tie. Domain 4 task 4.1 uses token price. Domain 1 task 1.2 uses the context window as a model axis. Domain 5 task 5.2 uses token count for truncated output.

Exam trap. Context overflow drops the end of a long input. The output looks cut off or ignores the last section.

Lesson 2 · Inference

First principle. Inference is the step where a trained model reads input and produces output. Each call has two phases. The model first reads the whole input. It then writes the output one token at a time.

Time to first token depends mostly on the input length and the queue. Tokens per second depends on the model and capacity. Streaming sends each token as it appears, which improves perceived speed. It does not shorten the total generation time. Three AWS modes serve inference. On-demand charges per token and has no commitment. Provisioned Throughput reserves model units for a steady rate. Batch inference runs offline from S3 to S3 at a lower price for select models.

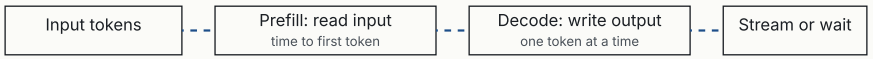


Diagram: inference has a read phase and a write phase. Built in SVG for this treatise. Schematic, not official.

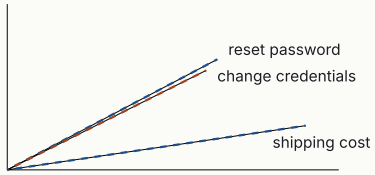
Exam tie. Domain 2 task 2.2 chooses the mode. Domain 4 task 4.2 measures time to first token and throughput. Domain 4 task 4.1 uses the mode as a cost lever.

Exam trap. Buying Provisioned Throughput does not change the speed of one call. It changes the reserved capacity. The call must use the provisioned model ARN.

Lesson 3 · Embeddings and similarity

First principle. An embedding maps text to a list of numbers, called a vector. Texts with similar meaning get vectors that point in similar directions. Similarity is a score that compares two vectors.

Titan Text Embeddings v2 outputs 1024, 512, or 256 dimensions. More dimensions hold more detail and cost more storage and compute. Cohere Embed separates queries from documents with an input type value. Use search_query for questions and search_document for passages. Embedding calls use InvokeModel with an embedding model. Converse does not produce embeddings.



Vectors point the same way when meaning matches.

Diagram: three vectors in a plane. Two requests with similar meaning point the same way. Built in SVG for this treatise. Schematic, not official.

Exam tie. Domain 1 task 1.4 and task 1.5 test the dimension choice and the input type. Domain 2 task 2.3 tests the API choice.

Exam trap. A bigger embedding model does not fix a bad chunk. The chunk must hold one complete answer.

Lesson 4 · Vector search and reranking

First principle. Vector search finds the stored vectors closest to a query vector. It returns the top-k results. Closest is not the same as most relevant. A reranker scores each candidate against the question.

Hybrid search adds keyword matching to vector search. Keyword search finds exact codes and citation strings. Vector search finds related meaning. Metadata filters narrow the search. Check the store documentation for when the filter applies.

Exam tie. Domain 1 task 1.4 selects the store. Domain 1 task 1.5 selects hybrid search, reranking, and query rewrite.

> **Exam trap.** Reranking fixes order. It does not add a chunk that retrieval missed.

Lesson 5 · Retrieval-augmented generation (RAG)

First principle. A model answers from its weights, which are fixed at training time. RAG adds facts at query time. The system retrieves text from your documents and places it in the prompt. The model answers from that text and cites it.

RAG has two paths. The ingest path runs ahead of time. It chunks, embeds, and stores documents. The query path runs per request. It embeds the question, retrieves chunks, optionally reranks, and generates an answer with citations. Knowledge Bases runs both paths for you.

The RAG diagram in Part 2 shows both paths. Read it after this lesson.

Exam tie. Domain 1 tasks 1.4 and 1.5 cover RAG. Domain 3 task 3.1 uses grounding checks on the answer. Domain 5 task 5.1 splits retriever metrics from generator metrics.

> **Exam trap.** Fine-tuning does not add changing facts. Use RAG for facts that change.

Lesson 6 · Chunking

First principle. A chunk is the unit of text that retrieval returns. A chunk must hold one complete answer. Too large, and the chunk adds noise. Too small, and the chunk loses context.

Fixed-size chunking works for short, self-contained text such as FAQs. Hierarchical chunking uses small child chunks for matching and larger parent chunks for context. It suits manuals and contracts. Semantic chunking splits on meaning and costs more to process. Custom chunking with Lambda keeps tables and figures whole. The chunking strategy is set when the data source is created. To change it, create a new data source and sync it.

Exam tie. Domain 1 task 1.3 selects the chunking strategy. Domain 5 task 5.2 traces quality drops to a changed chunking setting.

> **Exam trap.** A table split across chunks can lose its values. A chunk can look correct alone and still be wrong.

Lesson 7 · Customization: from prompts to LoRA and distillation

First principle. Customization changes the model or its input. Each rung changes more and costs more. Start with the cheapest rung. Stop when the requirement is met.

Prompt engineering changes only the instructions. RAG adds facts at query time. Fine-tuning changes the model weights with labeled examples. LoRA is a parameter-efficient method. It freezes the base weights and trains small adapter matrices. The update is the product of two small matrices, so the adapter is much smaller than the base model. You can store several adapters and load the one that fits the task. Distillation trains a smaller student model to copy a larger teacher model on one task. The student is cheaper and faster for that task.

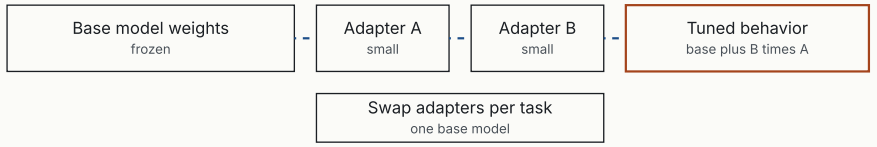


Diagram: LoRA keeps the base weights frozen and trains a small adapter. Built in SVG for this treatise. Schematic, not official.

Exam tie. Domain 1 task 1.2 covers the customization ladder. Domain 2 task 2.2 covers SageMaker AI deployment of a custom model. Domain 3 task 3.4 covers model documentation for a tuned model.

> **Exam trap.** LoRA does not fix facts that change. A tuned adapter still holds the facts from its training data. Use RAG for changing facts.

Lesson 8 · Agents and tools

First principle. An agent is a loop. The model reasons about the next step. It calls a tool. It observes the result. It repeats until a stop condition is met. A loop needs limits, because each step costs time and money.

A tool has a schema, which names its inputs. An action group in Bedrock Agents links the agent to a Lambda function. AgentCore Gateway exposes tools through MCP. Each tool call can pass through Policy in AgentCore before it runs. Trace records each step, the chosen tool, and the observation.

Exam tie. Domain 2 task 2.1 covers the agent loop, action groups, and aliases. Domain 2 task 2.2 covers AgentCore. Domain 3 task 3.1 covers the policy check.

Exam trap. A prompt that says "stop after three steps" is not a control. Enforce the limit in the workflow or the tool layer.

Lesson 9 · Safety layers

First principle. No single control stops every failure. Each layer checks something different. Input checks run before the model. Output checks run after it. Data controls protect storage and access.

Guardrails apply six policy types on input and output. Contextual grounding checks test whether the answer is supported by the source. The prompt attack filter detects jailbreak and injection patterns on input. Comprehend detects PII in text. Macie finds sensitive data in S3 at rest. KMS encrypts data at rest. PrivateLink keeps API traffic inside the VPC.

Exam tie. Domain 3 tasks 3.1 and 3.2 cover these layers. Domain 3 task 3.3 covers the audit side.

Exam trap. A denied topic is not a prompt attack detector. Use the prompt attack filter for attacks.

Lesson 10 · Evaluation

First principle. An evaluation compares outputs with a standard. The standard can be a known answer, a rubric, or a human judgment. The method must match the claim.

Automatic metrics suit facts with known answers. An LLM judge scores qualities such as faithfulness at scale. A judge must be calibrated against human ratings first. A golden set holds trusted prompts and answers. Regression suites rerun the golden set on each change. Canary releases send a small share of traffic to a new version.

Exam tie. Domain 5 tasks 5.1 and 5.2 cover evaluation methods, golden sets, and release gates.

Exam trap. An uncalibrated judge gives opinions, not measurements.

Lesson 11 · Operations signals

First principle. A production system emits three kinds of signal. Metrics count things over time. Traces follow one request across services. Logs hold the payloads. Each signal answers a different question.

CloudWatch metrics track tokens, invocations, and throttles. Anomaly detection builds a band from history and alarms when a value leaves it. X-Ray traces requests and downstream calls. Invocation logs hold prompts and completions. Logs Insights searches them. Model Monitor watches drift in structured features, not free-text quality.

Exam tie. Domain 4 task 4.3 covers monitoring. Domain 5 task 5.2 covers debugging.

Exam trap. Metrics alone cannot rebuild what the model received. Keep the payload logs under access control.

PART 1C · SERVICE DEEP DIVES

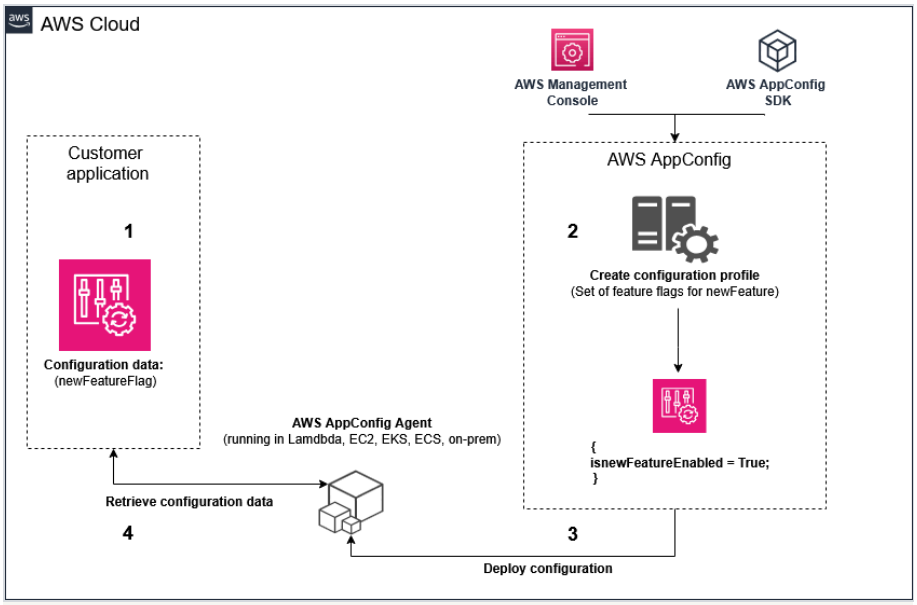
AWS AppConfig and Amazon SageMaker AI, from first principles

Both services appear in many questions. AppConfig controls how a running application reads its settings. SageMaker AI hosts and prepares models. Read each service from the purpose, then the parts, then the exam tie.

AWS AppConfig

First principle. A running application reads its settings from somewhere. If a setting changes, the application must pick up the change without a code release. AppConfig is the place that holds the settings and rolls them out.

Parts. A configuration profile defines where the settings live. The stores are the AppConfig hosted configuration store, AWS Secrets Manager, Systems Manager Parameter Store, and Amazon S3. A deployment strategy sets how fast a change reaches the application. It can roll out in steps over a period of time. Validators check the configuration before deployment. They check syntax and meaning. Alarms watch CloudWatch metrics during a deployment. If an alarm fires, AppConfig rolls the change back. Feature flags let you release a capability to some users first.



Official AWS image. Source: What is AWS AppConfig. Use for the deployment flow.

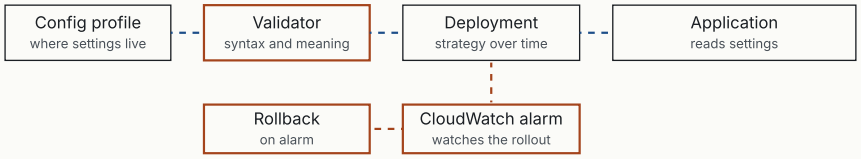


Diagram: a configuration change passes a validator, rolls out over a strategy, and rolls back on an alarm. Built in SVG for this treatise. Schematic, based on the official overview.

Model switch pattern. Store the model ID in a hosted configuration profile. The Lambda function reads the ID at runtime. A change to the ID rolls out with a deployment strategy. A CloudWatch alarm on errors rolls it back. No code release is needed. This is the pattern in Domain 1 task 1.2.

Exam trap. AppConfig controls settings. It does not route traffic to a model by prompt complexity. It does not host the model.

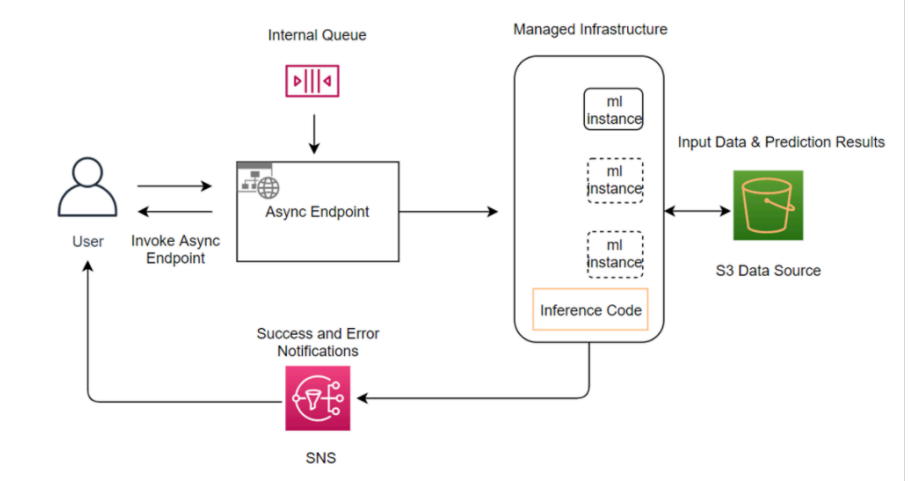
Amazon SageMaker AI

First principle. A custom or fine-tuned model needs a place to run, a way to track versions, and a way to watch its behavior. SageMaker AI covers those three needs. Read its parts in that order.

Hosting. A real-time endpoint serves steady traffic with autoscaling. A serverless endpoint serves intermittent traffic and scales to zero. Its memory runs from 1 GB to 6 GB. Provisioned Concurrency keeps capacity warm for predictable bursts. The first request after idle time can have a cold start. An asynchronous endpoint queues long requests. Its payload comes from S3. The result goes to S3. It supports processing times up to one hour and near real-time latency.

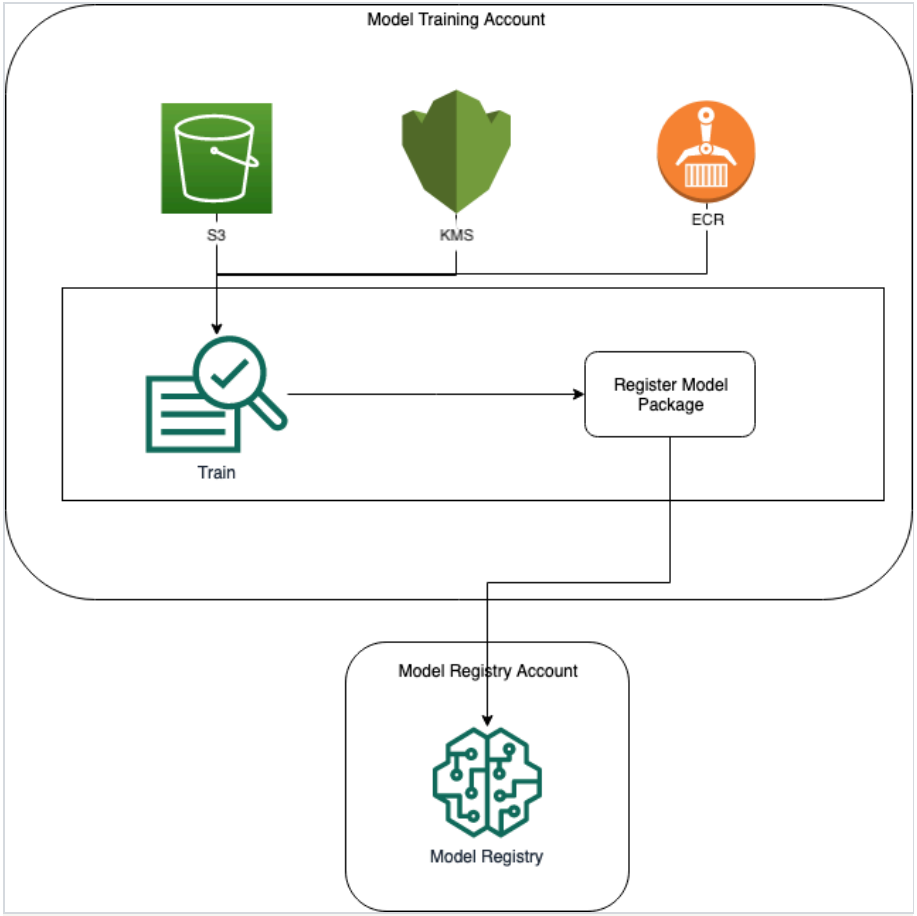


Official AWS image. Source: SageMaker AI serverless endpoints.



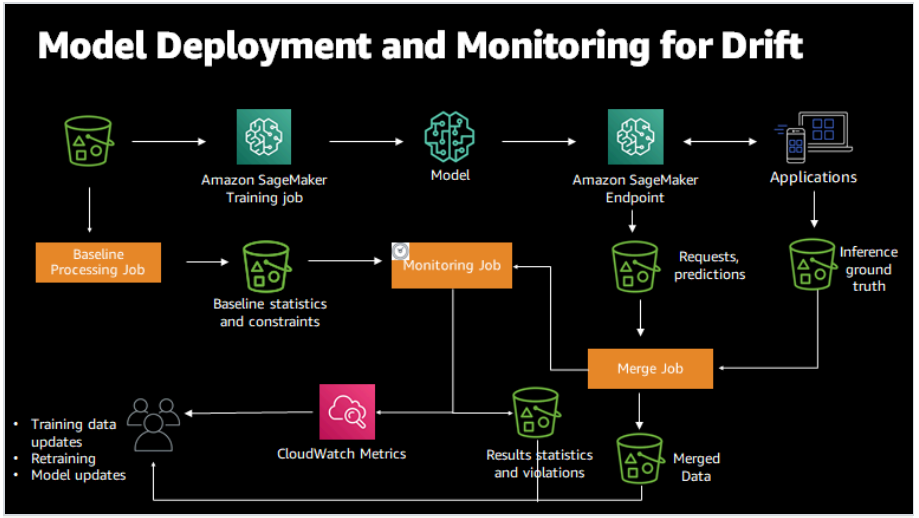
Official AWS image. Source: SageMaker AI asynchronous inference.

Registry. The model registry groups model versions. Each version has an approval status. A registry can share versions across accounts. Pipelines can require approval before deployment. A rollback points the endpoint to an earlier approved version.

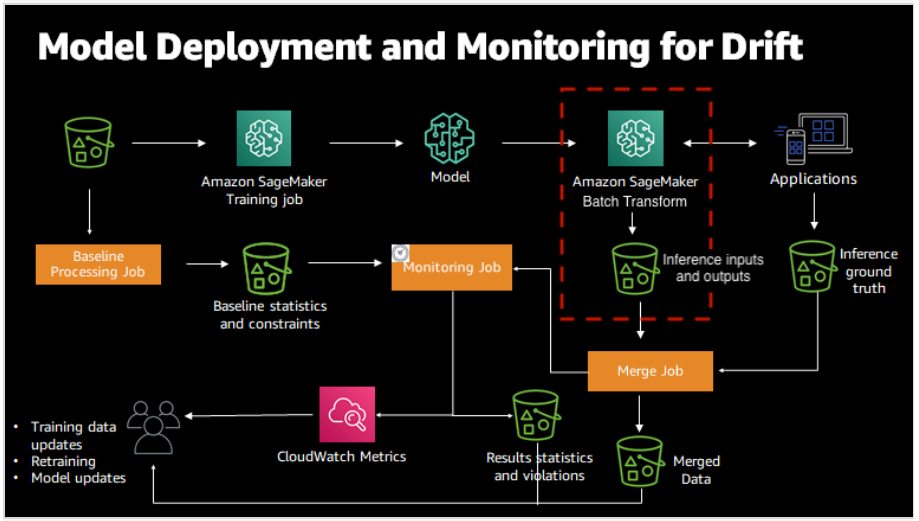


Official AWS image. Source: SageMaker Model Registry versions. Use for sharing versions across accounts.

Monitoring. Model Monitor watches drift in structured features and predictions. It does not score free-text answer quality. The SageMaker documentation states that Model Monitor is no longer open to new customers. Check that status before you rely on it.



Official AWS image. Source: SageMaker AI Model Monitor. Use for the monitoring flow.



Official AWS image. Source: SageMaker AI Model Monitor.

Customization. SageMaker AI fine-tunes a model. LoRA trains small adapters on top of frozen base weights. Distillation trains a smaller student from a teacher. Register each tuned version. Deploy it to an endpoint. Track the approval status.

Exam trap. SageMaker AI endpoints host models. Bedrock hosts the managed foundation models. Pick by who owns the model.

PART 1D · TIE-IN

How the lessons and services connect to the domains

Idea	Lesson	Service	Domain and task
Inference modes and cost	2	Bedrock on-demand, Provisioned Throughput, Batch	2.2, 4.1, 4.2
Tokens and context window	1	Bedrock Converse, CountTokens	1.2, 4.1, 5.2
Embeddings and vector search	3, 4	Titan Text Embeddings v2, Cohere Embed, OpenSearch Serverless, S3 Vectors	1.4, 1.5
RAG and citations	5, 6	Knowledge Bases, RetrieveAndGenerate	1.5, 5.1
LoRA and distillation	7	SageMaker AI fine-tuning, Model Registry, endpoints	1.2, 2.2, 3.4
Agents and tools	8	Bedrock Agents, AgentCore Gateway, Policy, Memory	2.1, 2.2, 3.1
Safety layers	9	Guardrails, Comprehend, Macie, KMS, PrivateLink	3.1, 3.2
Evaluation and judges	10	Model evaluation jobs, golden sets, canary releases	5.1, 5.2
Runtime configuration and rollback	1 and 11	AppConfig, CloudWatch alarms	1.2, 4.3, 5.2
Monitoring signals	11	CloudWatch, X-Ray, invocation logs, Model Monitor	4.3, 5.2

Read the table from left to right. A question that names one idea points to one lesson, one service, and one task.

PART 1E · VISUAL UNITS

Visual units for the components and ideas

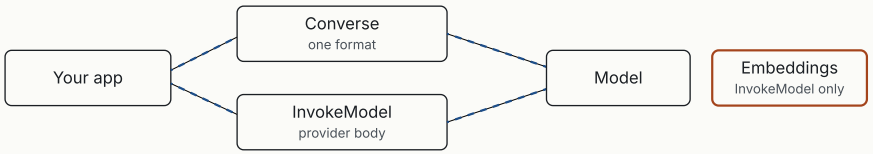
Each unit explains one idea. Units 1 to 20 cover components. Units 21 to 68 cover the domain tasks. Practical units p1 to p11 in Part 1F give the console and API steps. Read one unit at a time. Each unit starts with the first principle, then the picture, then the exam tie. Every diagram in this part is a schematic drawn from the official behavior. Official images appear in the component parts.

How to read a unit. Look at the picture first. Read the first principle second. Then check the trap.

VISUAL UNIT 1

Converse and InvokeModel: two doors to one model

First principle. Your application sends a request. Converse uses one request format for chat and tools across models. InvokeModel sends the body that one provider expects.



- Use Converse for chat, tool use, and multimodal input on supported models.
- Use InvokeModel for embeddings and provider-specific fields.
- A Claude body needs anthropic_version and max_tokens when you call InvokeModel.

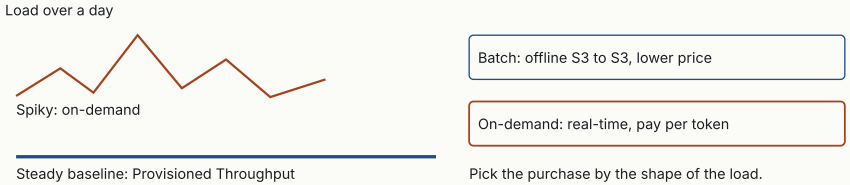
Exam tie. Domain 2 task 2.3 and Domain 1 task 1.2 test this choice. Embeddings always use InvokeModel.

Exam trap. Converse does not produce embeddings. A Titan body sent to Claude fails.

VISUAL UNIT 2

On-demand, Provisioned Throughput, and batch: three load shapes

First principle. Capacity is a shape problem. Spiky, steady, and offline load need different purchases. Match the purchase to the shape of the traffic.



- On-demand pays per token and has no commitment. It suits spiky or unknown traffic.
- Provisioned Throughput reserves model units. It suits a steady baseline.
- Batch runs offline from S3 to S3. It suits work that can wait for hours.

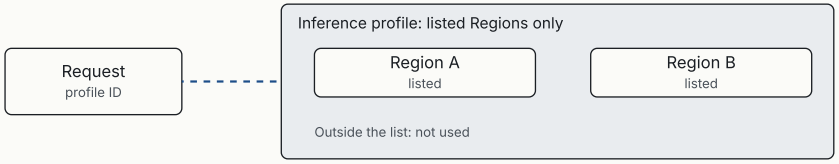
Exam tie. Domain 2 task 2.2 and Domain 4 task 4.1. Mixed traffic uses a reserved baseline and on-demand spikes.

Exam trap. More model units do not help a call that still uses the base model ID.

VISUAL UNIT 3

Cross-Region inference profiles

First principle. A profile ID names a set of Regions. A request that uses the profile can run in any Region in that set. The profile spreads load. It does not keep data in one place.



- Call the profile ID, such as one with the us. or eu. prefix.
- A geographic profile keeps routing inside its listed Regions.
- A global profile can route outside a strict residency boundary.

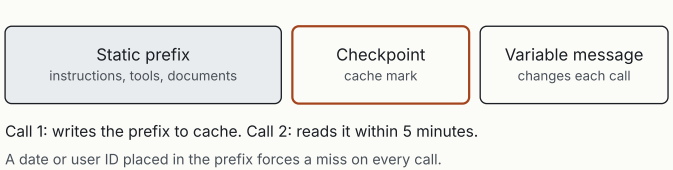
Exam tie. Domain 1 task 1.2 and Domain 4 task 4.1. Domain 3 task 3.2 covers data scope.

Exam trap. Cross-Region inference is not automatic failover for your app. Add retries and fallback in the application.

VISUAL UNIT 4

Prompt caching: the stable prefix pays once

First principle. A model reads the prompt from the start. If the start is the same as an earlier call, the system can reuse that work. The reused part must match exactly and sit before the changing text.



- Mark the static prefix with a cache checkpoint.
- Keep the user message at the end.
- The default time to live is 5 minutes.

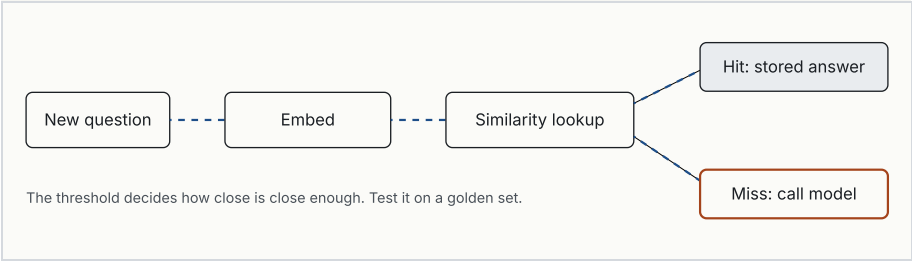
Exam tie. Domain 4 task 4.1 and Domain 2 task 2.5. The API marks the checkpoint with a cachePoint block. Caching lowers the cost of a repeated prefix.

Exam trap. A changing value inside the prefix breaks the cache. Move it to the end.

VISUAL UNIT 5

Semantic cache: match the meaning, not the words

First principle. A semantic cache stores answers next to the embedding of each question. A new question is embedded. If a stored question is close enough, the stored answer returns. Otherwise the model answers and the pair is stored.



- Hit: return the stored answer. No model call.
- Miss: call the model, then store the answer.
- Works when many questions repeat with different words.

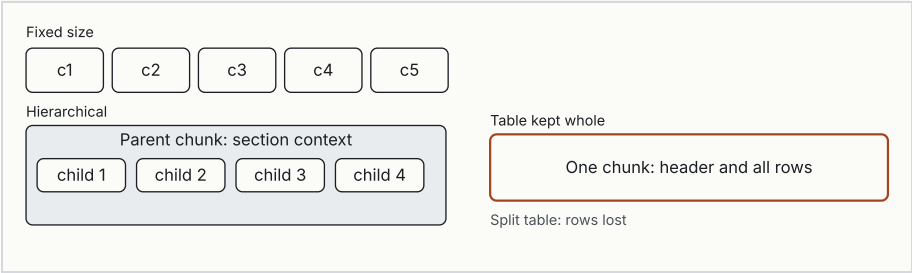
Exam tie. Domain 4 task 4.1 and Domain 4 task 4.2. Embeddings come from Domain 1 task 1.4.

Exam trap. Personal or unique questions rarely hit. A semantic cache saves little for them.

VISUAL UNIT 6

Chunk shape: fixed, hierarchical, and whole tables

First principle. A chunk is one piece of a document that retrieval can return. A chunk must hold one complete answer. The shape of the document decides the shape of the chunk.



- Fixed size suits FAQs and short text.
- Hierarchical keeps small child chunks for matching and larger parents for context.
- Custom Lambda chunking keeps a table or figure whole.

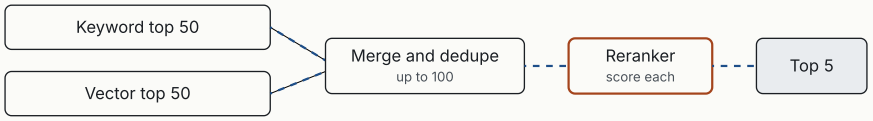
Exam tie. Domain 1 task 1.3. Domain 5 task 5.2 traces quality drops to chunking.

Exam trap. A table split across two chunks loses its values. Each half can look correct alone.

VISUAL UNIT 7

Hybrid search and reranking: a funnel

First principle. Retrieval returns many candidates. Each search method finds different candidates. Merge the lists. Then score each candidate against the question and keep the best few.



Fewer chunks reach the model. Each chunk adds tokens and cost.

- Keyword search finds exact codes and citation strings.
- Vector search finds related meaning.
- The reranker orders candidates by relevance. It does not add missing chunks.

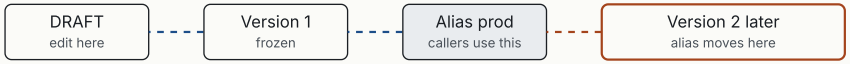
Exam tie. Domain 1 task 1.5. Domain 5 task 5.1 measures retriever quality.

Exam trap. Similarity is not relevance. Rerank before generation.

VISUAL UNIT 8

Agent versions: DRAFT, version, alias

First principle. Editing an agent changes DRAFT. Callers do not use DRAFT. A version is a frozen snapshot. An alias points to one version. Callers use the alias, so you can move traffic without changing the caller.



Callers send agentId and agentAliasId. Neither ID changes when the version changes.

- Edit DRAFT, then prepare the agent.
- Create a version from the prepared DRAFT.
- Point the alias to the new version.

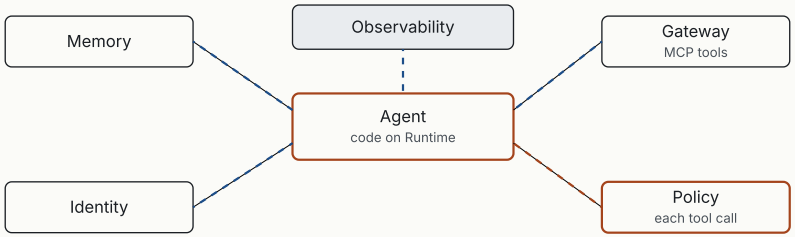
Exam tie. Domain 2 task 2.1. Domain 5 task 5.2 covers the old-behavior symptom.

Exam trap. A passing console test on DRAFT does not prove the alias works.

VISUAL UNIT 9

AgentCore components around one agent

First principle. An agent needs a place to run, memory, tools, identity, rules, and traces. AgentCore puts those parts behind one service. Each part answers one question about the agent.



- Runtime hosts the agent code from any framework.
- Memory keeps context. Gateway exposes tools as MCP tools.
- Identity decides who can call. Policy checks each tool call. Observability records traces.

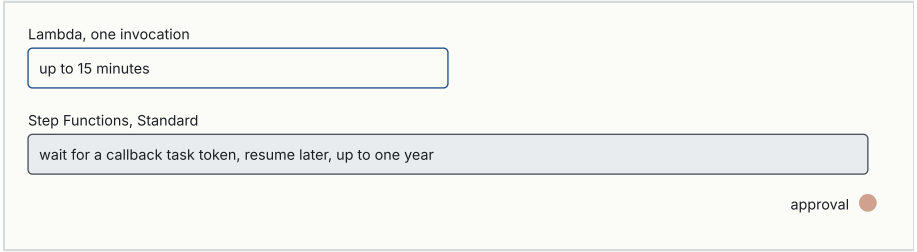
Exam tie. Domain 2 task 2.2. Domain 3 tasks 3.1 and 3.2. Domain 4 task 4.3.

Exam trap. AgentCore is not a model. It hosts and governs the agent around the model.

VISUAL UNIT 10

Lambda limit versus a waiting workflow

First principle. A Lambda invocation ends after at most 15 minutes for a standard function. A wait for a person can last days. A durable workflow stores its state and resumes later.



- Keep each Lambda step short.
- Step Functions Standard keeps state for waits and runs up to one year.
- A callback task token pauses the flow until a person or system responds.

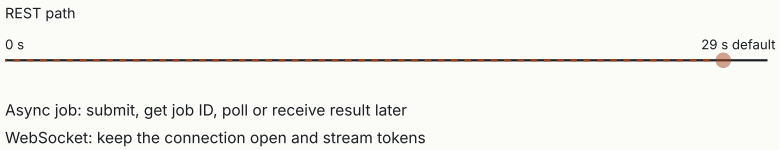
Exam tie. Domain 2 task 2.2 and Domain 1 task 1.6.

Exam trap. A Lambda that waits for a person pays for the wait and hits the limit.

VISUAL UNIT 11

API Gateway timeout versus a stream

First principle. An API Gateway REST integration has a default timeout of 29 seconds. A slow model call that runs longer loses its connection at that point. A stream that stays open can carry tokens for longer.



- Raising the Lambda timeout does not change the gateway limit.
- WebSocket APIs keep a connection open for streams.
- An asynchronous job returns a job ID, then the client polls or receives the result.

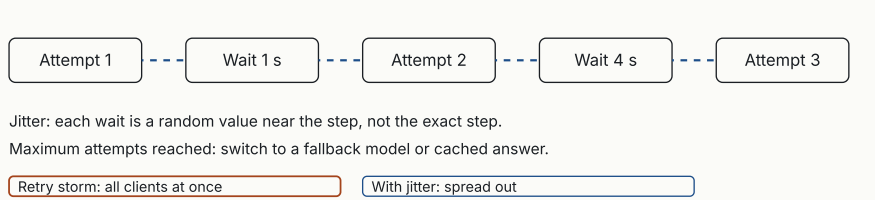
Exam tie. Domain 2 tasks 2.3 and 2.4.

Exam trap. Streaming improves perceived speed. It does not extend a REST timeout.

VISUAL UNIT 12

Retry with backoff and jitter

First principle. A failed call can succeed later. Retrying at once adds load to a service that is already busy. Waiting longer after each failure spreads the load. Random jitter stops many clients from retrying at the same moment.



- Wait longer after each failure.
- Add random jitter to each wait.
- Stop after a maximum number of attempts, then fall back.

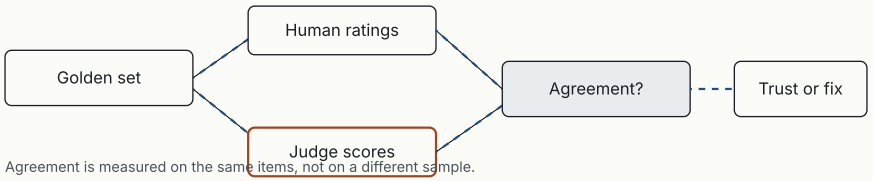
Exam tie. Domain 2 task 2.4. Domain 4 task 4.1 covers quotas and throttling.

Exam trap. A tight retry loop makes an overload worse. It is never the correct answer.

VISUAL UNIT 13

Judge calibration before trust

First principle. An LLM judge gives a score. A score means something only when it agrees with people. Rate a fixed golden set with humans. Run the judge on the same set. Compare the two.



- Low agreement means the judge is not valid for the claim.
- Pin the judge model ID. A new judge gives new scores.
- Human review covers tone and voice.

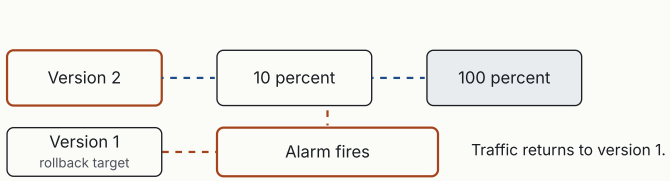
Exam tie. Domain 5 task 5.1. Domain 3 task 3.4 uses judges for fairness checks.

Exam trap. An uncalibrated judge gives opinions, not measurements.

VISUAL UNIT 14

Canary and rollback: release in steps

First principle. A new version can fail on real users. A canary sends a small share of traffic to it first. Metrics decide the next step. If a metric breaks, traffic returns to the last approved version.



- Start at a small share, such as 10 percent.
- Move traffic only when the alarms stay quiet.
- Rollback points traffic to the last approved version.

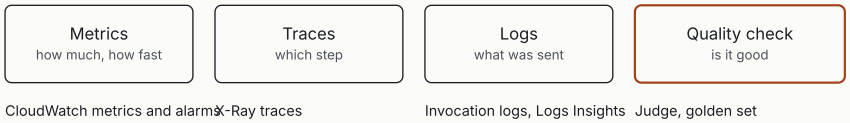
Exam tie. Domain 5 tasks 5.1 and 5.2. Domain 2 task 2.3 covers AppConfig and aliases.

Exam trap. A re-sync or a prompt edit is a release. It needs the same gate.

VISUAL UNIT 15

Four monitoring signals, four questions

First principle. Each signal answers one question. Metrics count. Traces follow one request. Logs hold payloads. Quality checks score answers over time.



- Metrics: is it slow or expensive?
- Traces: which step is slow?
- Logs: what did the model receive? Judge: is the answer good?

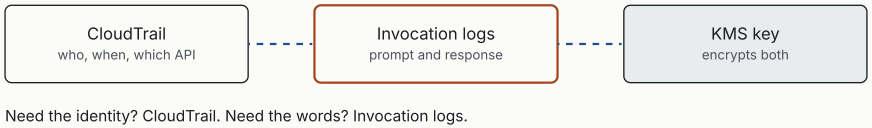
Exam tie. Domain 4 task 4.3. Domain 5 task 5.2.

Exam trap. Metrics alone cannot rebuild a wrong answer. Keep the payloads.

VISUAL UNIT 16

Who called versus what was sent

First principle. CloudTrail records who called which API, when, and from where. Invocation logs record the prompt and the response. These are two different records. Each one answers a different audit question.



- CloudTrail answers identity and timing.
- Invocation logs answer content.
- Encryption with KMS protects both at rest.

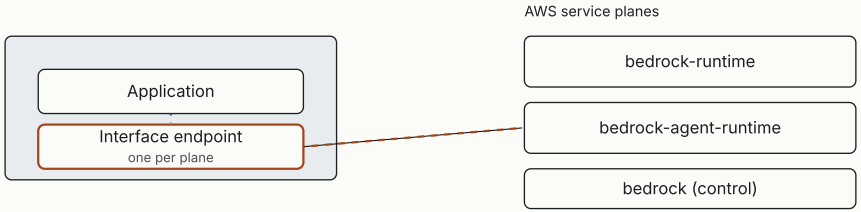
Exam tie. Domain 3 task 3.2 and task 3.3.

Exam trap. CloudTrail does not show row content. Invocation logs show the original request even when output is masked.

VISUAL UNIT 17

PrivateLink and the VPC boundary

First principle. A VPC endpoint gives private IP access to an AWS service. Traffic from the VPC stays on the AWS network. Each Bedrock plane has its own endpoint.



- bedrock-runtime serves model invocations.
- bedrock-agent and bedrock-agent-runtime serve agents.
- bedrock serves the control plane.

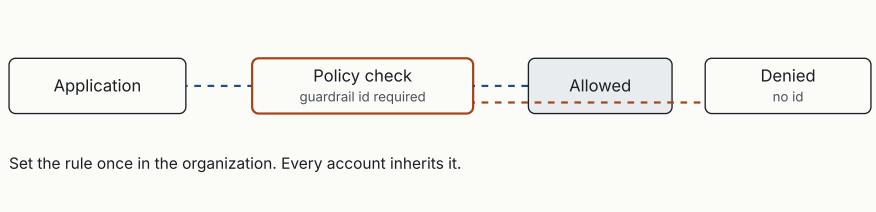
Exam tie. Domain 3 task 3.2.

Exam trap. One runtime endpoint does not cover agent calls. Check each plane.

VISUAL UNIT 18

Organization rules with SCPs and condition keys

First principle. A service control policy sets the most a member account can do. A condition key limits one action. A condition on the guardrail identifier can make the guardrail mandatory on each call.



- The IAM layer applies even if application code changes.
- A call without the guardrail identifier is denied.
- This is a rule across accounts, not a setting in one app.

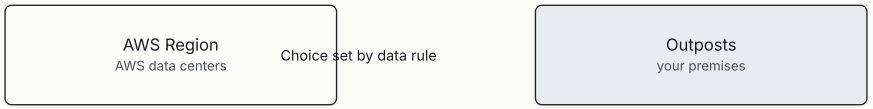
Exam tie. Domain 3 task 3.3.

Exam trap. A guardrail that only the app calls can be skipped. Enforce it in IAM.

VISUAL UNIT 19

Outposts versus a Region

First principle. A Region runs in AWS data centers. Outposts runs AWS services on your premises. Data processed on Outposts stays at the site for the rule that requires it.



Both use the same AWS APIs. The location of the data changes.

- Use a Region for most workloads.
- Use Outposts when a rule says the data stays on site.
- The same AWS programming interfaces apply on Outposts.

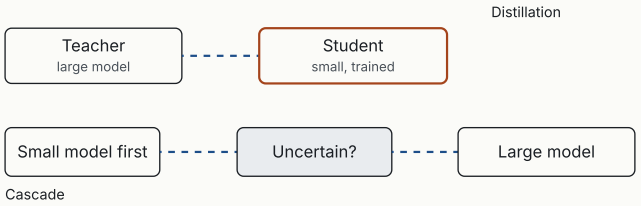
Exam tie. Domain 2 task 2.3.

Exam trap. A Region in the cloud does not keep data on your premises.

VISUAL UNIT 20

Distillation and cascade routing

First principle. A large model is accurate and costly. A small model is cheaper and faster. Distillation trains the small model to copy the large one on one task. Cascade routing tries the small model first and escalates only when needed.



- Distillation builds a cheaper student for a narrow task.
- Cascade escalates on uncertainty or complexity.
- Routing needs a signal. Without a signal the router cannot decide.

Exam tie. Domain 1 task 1.2. Domain 4 task 4.1 and task 4.2.

Exam trap. Sending every request to the largest model wastes money.

Which unit explains which component

Component or idea	Unit
Converse, InvokeModel	Unit 1
Provisioned Throughput, on-demand, batch	Unit 2
Cross-Region inference profiles	Unit 3
Prompt caching	Unit 4
Semantic cache	Unit 5

Chunking	Unit 6
Hybrid search, reranker	Unit 7
Bedrock Agents DRAFT, version, alias	Unit 8
AgentCore components	Unit 9
Lambda, Step Functions	Unit 10
API Gateway	Unit 11
Retries and throttling	Unit 12
Judge calibration	Unit 13
Canary and rollback, AppConfig	Unit 14
CloudWatch, X-Ray, logs, judge	Unit 15
CloudTrail and invocation logs, KMS	Unit 16
PrivateLink	Unit 17
SCP and IAM condition keys	Unit 18
Outposts	Unit 19
Distillation, cascade routing	Unit 20

PART 1F · PRACTICAL CONSOLE AND API STEPS

What to do in the console and the API

Each unit lists the steps that the official page states. Each step cites its page. The exam tests whether you know where a setting lives and which setting fixes a symptom. A step marked Not in source does not appear in the page this build could read. Check that step on the live console before exam day.

> Screenshots below are official AWS documentation images. No account screenshots appear in this treatise.

PRACTICAL UNIT 1

Create a knowledge base from a data source

Where. Source: Create a knowledge base by connecting to a data source.

1. In the left navigation pane, choose Knowledge bases.
2. Choose an AWS Identity and Access Management (IAM) service role that gives Amazon Bedrock access to the knowledge base.
3. Choose the data source to connect the knowledge base to.
4. Choose an embeddings model. For multimodal data, the page names Amazon Titan Multimodal Embeddings G1 or Cohere Embed v3.
5. Choose a vector store. Use one you created, or let Bedrock create one.
6. Set the security configuration for the knowledge base.

API route. Sync the source with StartIngestionJob. Query with the retrieve and generate operations.

Not in source. The page does not list every field on each screen.

PRACTICAL UNIT 2

Test a knowledge base retrieval

Where. Source: Query a knowledge base and retrieve data.

1. In the left navigation pane, choose Knowledge bases.
2. Choose the knowledge base you want to test.
3. In the test window, enter a query in the text box and select the run action.
4. Read the retrieved chunks and their sources before you judge the answer.

API route. Use the retrieve operation to return chunks without generation.

Not in source. Not in source for the other console fields on this page.

PRACTICAL UNIT 3

Create and configure an agent manually

Where. Source: Create and configure agent manually.

1. Choose a service role. Create and use a new role, or use an existing role.
2. Add action groups to define the actions the agent can perform.
3. Add a knowledge base so the agent can search and query data.
4. Test the agent in the test window and troubleshoot its behavior.
5. For several agents, configure multi-agent collaboration.

API route. Invoke the agent with the invoke agent operation, using an agent alias ID.

Not in source. The fetched page does not show the prepare or alias steps. See Domain 2 for the alias rule.

PRACTICAL UNIT 4

Create a batch inference job

Where. Source: Create a batch inference job.

1. Choose a service role. Use an existing role, or create a custom service role for batch inference.
2. Create the batch inference job and choose the model.
3. Point the job at an input location in S3 and an output location in S3.

API route. Create the job with the create model invocation job operation.

Not in source. Not in source for the other console fields on this page.

PRACTICAL UNIT 5

Enable model invocation logging

Where. Source: Monitor model invocation using CloudWatch Logs and Amazon S3.

1. Choose an S3 destination. Create an S3 bucket for the logs.
2. Add a bucket policy to the bucket. Replace the example values with your own.
3. Or choose a CloudWatch Logs destination and set its log group.
4. Turn on invocation logging in the Bedrock settings. The fetched page does not give the screen path.

Not in source. The fetched page does not give the exact toggle name on the settings screen.

PRACTICAL UNIT 6

Create a prompt in Prompt management

Where. Source: Create a prompt using Prompt management.

1. Provide a name and an optional description for the prompt.
2. Write the prompt message. Use double curly braces for each variable, such as a variable name in double braces.
3. Set a system prompt when you need to give instructions or context.
4. Test the prompt. Setting a base inference parameter as an additional field does not override the value in the prompt configuration.

Not in source. Not in source for the other console fields on this page.

PRACTICAL UNIT 7

Turn on prompt caching in the console

Where. Source: Prompt caching for faster model inference.

1. Open the prompt caching setting in the console. The page shows the toggle as an image.
2. Place the stable content first. Place the changing content last.
3. Read the cache metrics panel that the page shows as an image.

Not in source. The fetched page shows the toggle as an image only. The menu path is Not in source.

PRACTICAL UNIT 8

Create a configuration and feature flags in AppConfig

Where. Source: Creating feature flags and free form configuration data in AWS AppConfig.

1. Create a configuration profile. Choose a feature flag or a free-form configuration.
2. Choose the store for free-form data. The options include the hosted store, Secrets Manager, Parameter Store, and Amazon S3.
3. Set the configuration profile IAM role when the store requires one.
4. Create multi-variant feature flags when you need variants for an experiment.

Not in source. Not in source for the other console fields on this page.

PRACTICAL UNIT 9

Create a deployment strategy in AppConfig

Where. Source: Working with deployment strategies.

1. Create a deployment strategy. Check the rollout field names on the console. The fetched page does not list them.
2. Attach the strategy to the deployment of the configuration.
3. Let the alarms watch the rollout. A firing alarm rolls the change back.

Not in source. Not in source for the other console fields on this page.

PRACTICAL UNIT 10

Create an asynchronous endpoint in SageMaker AI

Where. Source: How to create an Asynchronous Inference Endpoint.

1. Create a model with the create model operation.
2. Create an endpoint configuration with the create endpoint configuration operation. Set the asynchronous settings.
3. Create the HTTPS endpoint with the create endpoint operation.
4. Send the request payload to S3. Read the result from the S3 output location.

Not in source. Not in source for the other console fields on this page.

PRACTICAL UNIT 11

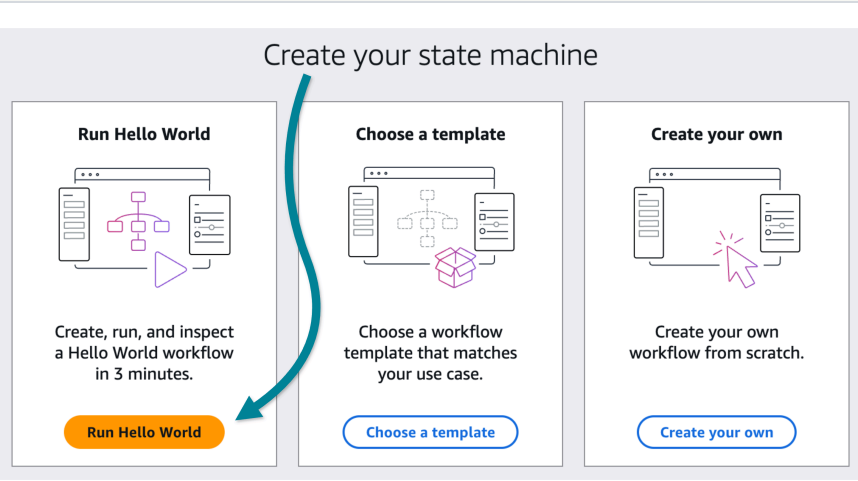
Build a Step Functions state machine from a template

Where. Source: Learn how to get started with Step Functions.

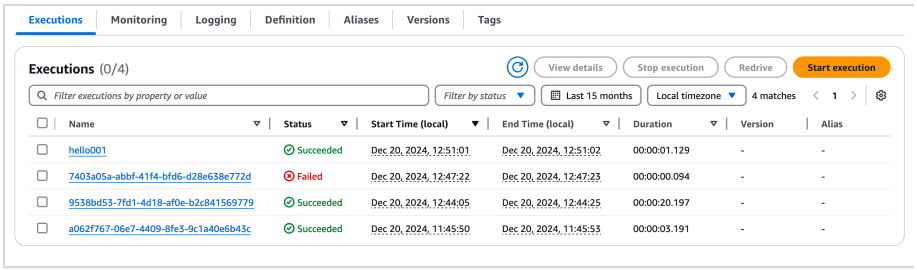
1 Open the Step Functions console and create a state machine from a template.
2 Select states on the canvas to review their settings.
3 Start an execution. Review the execution details and the state output.
4 Choose the IAM role for the state machine. Use the IAM console when the role needs changes.

Not in source. The fetched page does not show the callback task token steps. See Domain 2 unit 10.

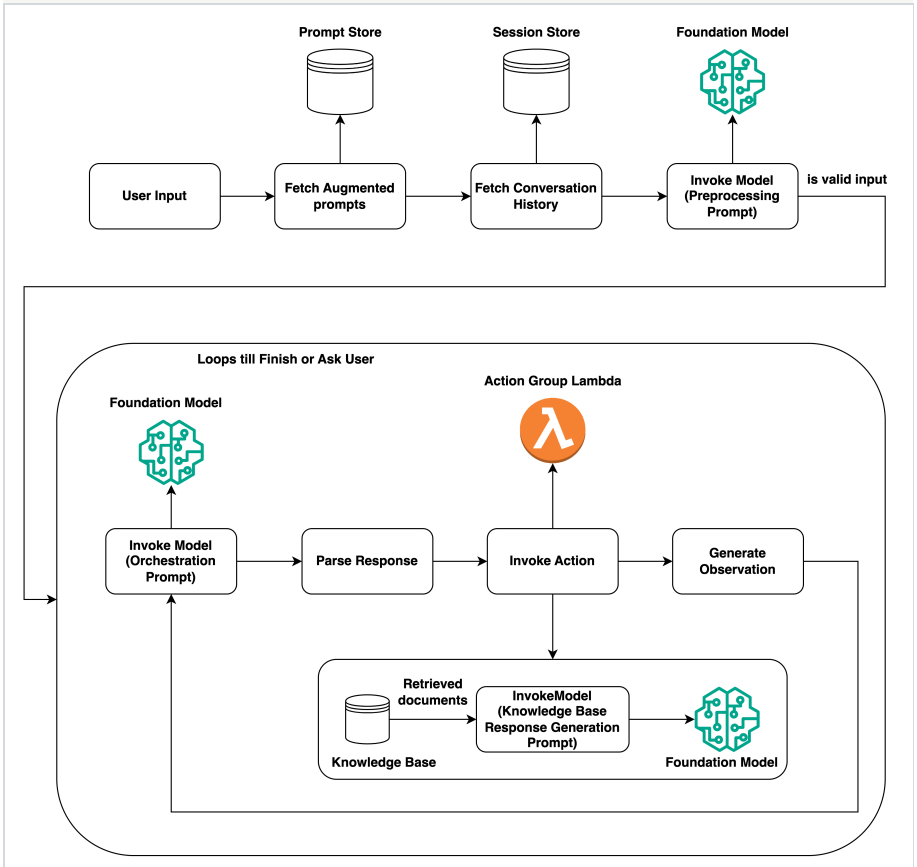
Screenshots from the official pages



Official AWS image. Source: Learn how to get started with Step Functions. Use for the template choice.



Official AWS image. Source: Learn how to get started with Step Functions. Use for execution review.



Official AWS image. Source: Amazon Bedrock Agents, how agents work.

Topics with no fetched console steps

These topics have no console steps in the pages this build could read. Each one is marked Not in source for steps.

Topic	Console or API steps
Provisioned Throughput purchase	Not in source
Guardrail creation steps (the page title is confirmed)	Not in source
SageMaker real-time endpoint creation (page stub)	Not in source
Lambda timeout setting	Not in source
API Gateway integration timeout setting	Not in source
Macie, Lake Formation, CloudTrail, KMS, and PrivateLink console steps	Not in source
Comprehend, Transcribe, Glue Data Quality, and Data Automation steps	Not in source
OpenSearch Serverless, S3 Vectors, Neptune, and DynamoDB TTL steps	Not in source
CloudWatch anomaly alarm and X-Ray steps	Not in source
Model Monitor and judge evaluation job steps	Not in source
Model Registry approval steps (the page shows approval status in the API output)	Not in source

PART 2 · DOMAIN 1 · WEIGHT 31 PERCENT (REPOSITORY README)

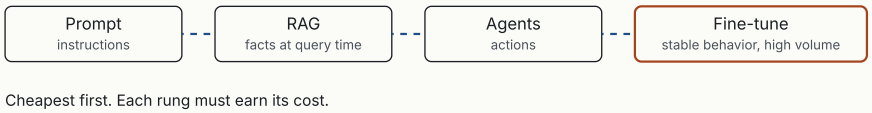
Foundation model integration, data management, and compliance

VISUAL UNITS FOR THIS DOMAIN

VISUAL UNIT 21

The approach ladder: cheapest rung first

First principle. Each rung adds more change and more cost. Prompts change instructions. RAG adds facts at query time. Agents add actions. Fine-tuning changes weights. Stop at the first rung that meets the requirement.



Cheapest first. Each rung must earn its cost.

- Start with prompt engineering.
- Add RAG when facts change or citations are needed.
- Add agents when the model must act. Fine-tune only for a stable, high-volume behavior.

Exam tie. Domain 1 task 1.1 and task 1.2.

Exam trap. Fine-tuning on documents that change is wrong. Use RAG.

VISUAL UNIT 22

Model choice as a constraint filter

First principle. A model choice is a filter. Remove each candidate that breaks one hard constraint. Keep the cheapest model that passes all of them.



Each filter removes candidates. One survivor is the answer.

- Check capability, cost per token, latency, context window, and Region.
- Test the final choice on your own data.
- The flagship model is not the default. It can break a latency budget.

Exam tie. Domain 1 task 1.2.

Exam trap. A model that is cheap but unavailable in your Region is not a choice.

VISUAL UNIT 23

Data pipeline: four stages before the model

First principle. A model reads what it receives. Bad data gives bad answers. Clean the data in stages. Each stage uses one tool for one job.



One tool per stage. Do not swap their jobs.

- Validate with Glue Data Quality rules.
- Process documents, audio, and video with Data Automation or Transcribe.
- Format, then enhance with Comprehend for PII.

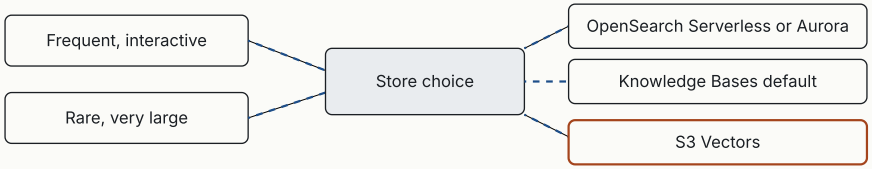
Exam tie. Domain 1 task 1.3.

Exam trap. Guardrails do not validate data quality or extract fields.

VISUAL UNIT 24

Vector store choice follows the data shape

First principle. A vector store holds vectors and answers nearest-neighbor queries. The right store depends on query rate, latency, and whether you already run a database.



- Knowledge Bases is the default for private documents with citations.
- S3 Vectors fits large corpora with infrequent queries.
- Neptune is for graph traversal, not plain document search.

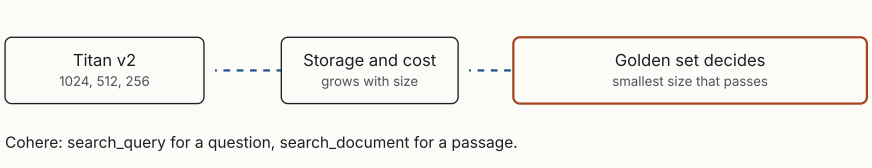
Exam tie. Domain 1 task 1.4.

Exam trap. ElastiCache is not a durable store of record.

VISUAL UNIT 25

Embedding dimensions and the input type

First principle. An embedding is a list of numbers. More numbers hold more detail and cost more storage. Retrieval also needs the right encoding for a query and for a document.



- Titan Text Embeddings v2 outputs 1024, 512, or 256 dimensions.
- Cohere Embed uses search_query for questions and search_document for passages.
- Test both sizes on a golden set.

Exam tie. Domain 1 tasks 1.4 and 1.5.

Exam trap. Do not guess the size. Measure it.

VISUAL UNIT 26

Prompt governance: templates in version control

First principle. A prompt changes behavior like code does. Store it, version it, test it, and roll it back. An untracked prompt has no rollback point.



- Store variables and roles in Prompt Management.
- Approve before publication. Track use in CloudTrail.
- Test with a golden set before each release.

Exam tie. Domain 1 task 1.6 and Domain 2 task 2.5.

Exam trap. A prompt that changes in place has no safe rollback point.

VISUAL UNIT 27

Prompt Flows versus Step Functions

First principle. A flow is a fixed chain of nodes. A workflow is a durable state machine. Pick the flow for predictable steps. Pick Step Functions for waits, retries, and human approval.



Predictable steps: flow. Waits and approval: Step Functions.

- Prompt Flows: console chain with branches.
- Step Functions: callback tokens and waits up to one year in Standard.
- A multi-day approval needs Step Functions Standard.

Exam tie. Domain 1 task 1.6.

Exam trap. A Lambda that waits for a person hits the 15-minute limit.

VISUAL UNIT 55

Structured output: a schema, not a request

First principle. A model writes text. A request for JSON is a hope. A schema is a contract. The response must match the schema before your code reads it.



- Structured output support depends on the model and the API. Check the API reference for the exact field.
- Check the model card before you rely on it.
- Validate the output anyway at the boundary.

Exam tie. Domain 1 task 1.3, Domain 3 task 3.1, and Domain 5 task 5.2.

Exam trap. A prompt that asks for JSON does not guarantee valid JSON.

VISUAL UNIT 56

Model lifecycle: every release is a version

First principle. A model changes over time. A registry gives each version a number and an approval status. A rollback needs a previous approved version to return to.



- Register each version.
- Approve before deployment.
- Keep the previous approved version for rollback.

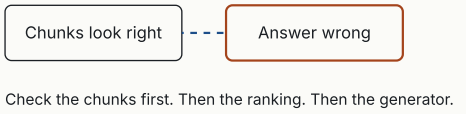
Exam tie. Domain 1 task 1.2. Domain 3 task 3.3 covers the audit trail.

Exam trap. A model or prompt that changes in place has no rollback point.

VISUAL UNIT 57

Retrieval failures: wrong chunks, not wrong model

First principle. An answer uses retrieved text. If the answer is wrong while the chunks look right, the fault is in how the chunks were built or ranked. Changing the generator model does not repair retrieval.



Check the chunks first. Then the ranking. Then the generator.

- Check whether the chunks split a complete answer.
- Check whether similarity matched meaning or only words.
- After a re-sync, check the chunking setting and the embedding model version.

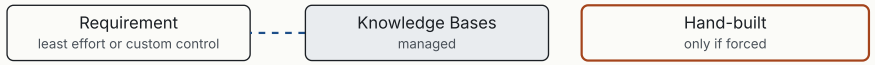
Exam tie. Domain 1 task 1.5 and Domain 5 task 5.2.

Exam trap. Changing the generator does not fix broken retrieval.

VISUAL UNIT 58

Knowledge Bases or hand-built RAG

First principle. Knowledge Bases runs the RAG pipeline for you. You sync sources, then call one API. Hand-built RAG gives you control over each stage. Choose hand-built only when a stated requirement needs that control.



- Least custom effort: Knowledge Bases.
- Custom retrieval algorithm: hand-built.
- Web sources: check connector support for your knowledge base type.

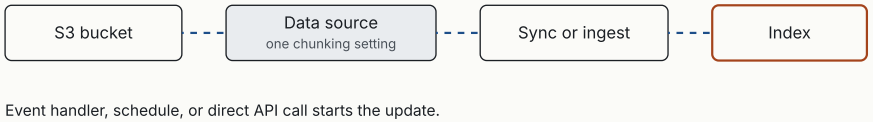
Exam tie. Domain 1 tasks 1.4 and 1.5.

Exam trap. A hand-built pipeline is wrong when the scenario asks for least effort.

VISUAL UNIT 59

Knowledge Base data sources: sync or ingest

First principle. A data source tells the knowledge base where its documents live. A sync reads the source and updates the index. Ingestion through the direct API adds, updates, or deletes single files. Check the chunking setting on the data source before a change.



Event handler, schedule, or direct API call starts the update.

- Sync on a schedule with a scheduler that calls StartIngestionJob.
- Ingest on an event with your own handler.
- Direct ingestion uses the KnowledgeBaseDocuments API for S3 and custom sources.

Exam tie. Domain 1 task 1.4.

Exam trap. To change chunking, create a new data source. Then sync it.

VISUAL UNIT 60

Citations: the audit trail for an answer

First principle. A citation links one answer span to the source chunk that supports it. A grounding score rates whether the answer is supported. Auditors sample both.



- Knowledge Bases can return citations with generated answers. Check the API reference for the fields.
- Contextual grounding returns a score per answer.
- Citations are attribution, not proof of truth.

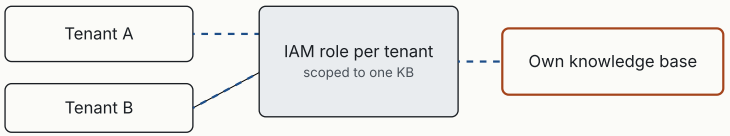
Exam tie. Domain 1 task 1.5, Domain 3 tasks 3.3 and 3.4.

Exam trap. A hand-built citation layer is wrong when the scenario asks for least effort.

VISUAL UNIT 61

Tenant isolation is structural

First principle. Tenant data must stay apart. A prompt that tells the model to ignore other tenants is not isolation. Isolation comes from the platform: separate knowledge bases, IAM roles, and metadata filters.



- One knowledge base for each tenant.
- IAM role scoped to its own knowledge base.
- Metadata filters for region or product line.

Exam tie. Domain 1 task 1.4 and Domain 3 task 3.2.

Exam trap. Prompt instructions are not isolation.

VISUAL UNIT 62

MCP tools: one standard for tool access

First principle. Model Context Protocol (MCP) is a standard for agents to find and call tools. One protocol replaces a custom client for each tool. A remote server uses Streamable HTTP.



- Tools announce what they do.
- Agents call tools through one client.
- MCP is not a model and not a guardrail.

Exam tie. Domain 1 task 1.5 and Domain 2 task 2.1.

Exam trap. MCP is the tool connection standard. It is not a safety control.

VISUAL UNIT 63

Service distractors: read what the service stores

First principle. Some services look right for a GenAI need. Each one stores or returns a different thing. Kendra is not open to new customers. Translate changes language, not PII. ElastiCache is a cache, not a durable store.



- Kendra: no longer open to new customers. Knowledge Bases is the documented alternative.
- Translate: translates text. It does not detect PII.
- ElastiCache: an in-memory cache. Check its durability settings before you use it as a store.

Exam tie. Domain 1 tasks 1.3 and 1.4.

Exam trap. Speed does not make a service durable.

Domain 1 asks one question. How do you get the correct model, the correct data, and the correct controls into a working system on AWS? It has six task statements. Read each one in order.

1.1 Design the solution first, then prove it

Design from the constraints. Then build a proof of concept. Use on-demand inference first. Buying Provisioned Throughput or hardware before a proof of concept is a distractor pattern. Prove the approach on real data, then commit capacity.

Use the AWS Well-Architected Framework and the Generative AI Lens to keep designs consistent across teams.

1.1 The approach ladder

Climb the ladder from the cheapest rung. Stop at the first rung that meets the requirement.

Rung	Use when	Signal words
1. Prompt engineering	Start here. The cheapest step.	Format, tone, instructions
2. RAG	Documents change often, or answers must cite sources.	Private, current, cite, sources
3. Agents	The model must take actions, such as calling tools or APIs.	Take action, call, update a system
4. Fine-tuning	Stable behavior and a large, repeated task volume.	Fixed format at high volume, distill

Exam trap. Fine-tuning a model on documents that change is wrong. Use RAG. Knowledge goes to retrieval. Behavior goes to weights.

1.1 Knowledge or behavior

Need	Approach
Private, current, or cited facts	RAG with a Knowledge Base
Fixed format, tone, or voice at high volume	Fine-tune a small model (distillation)
Calls to APIs or databases	Agents

1.2 Choose a model on five axes

Axis	What to check
Capability	Reasoning, multilingual, code, vision

Cost per token	Input and output prices differ. Output tokens often cost more.
Latency	Time to first token and tokens per second. Streaming improves perceived speed.
Context window	How much text fits in one call, including retrieved chunks.
Regional availability	Is the model offered in the Region where you run?

Order of decision: latency needs, then reasoning difficulty, then cost. Test the final choice on your own data. The flagship model is not the default answer. It can break a latency budget.

1.2 Routing mechanisms

Mechanism	Routes by	Use when
Cross-Region inference profile	Regional availability and throughput	More throughput across Regions. Same model family.
Intelligent prompt routing	Predicted response quality and cost	Chooses between two models in one model family.
Lambda routing with AppConfig	Rules you write	You need your own policy and a model ID read at runtime.

Exam trap. A cross-Region profile routes for throughput and availability. It does not choose a model by prompt complexity.

1.2 Cross-Region inference and data scope

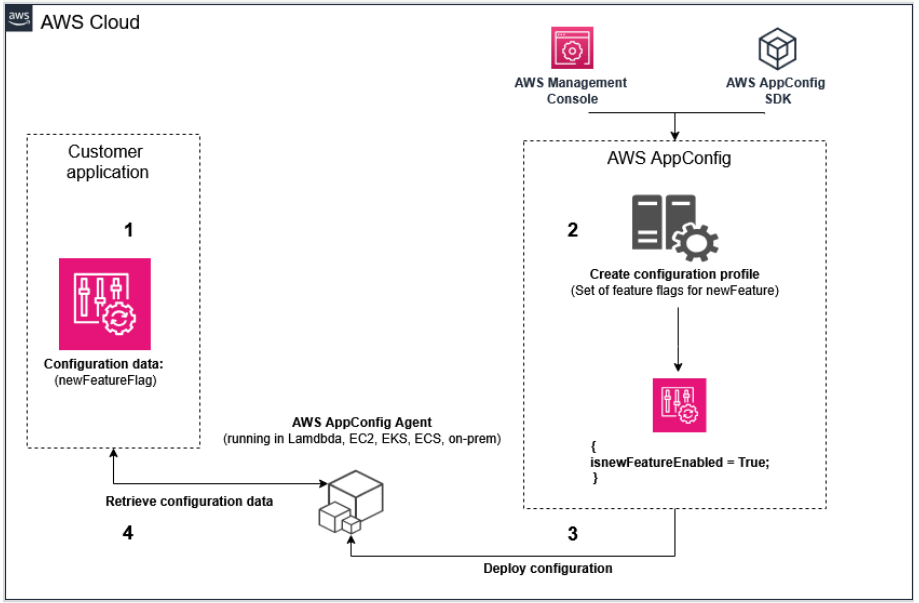
Call the inference profile ID, such as one with the `us.` or `eu.` prefix. Requests route to Regions listed in the profile.

A global profile can send requests outside a strict EU boundary. A geographic EU profile keeps routing inside the EU Regions it lists. Check the profile scope against the residency rule.

Cross-Region inference does not provide automatic failover for your application. Add retries and fallback in the application for an availability target.

1.2 Switch, survive, and customize

1. Switch without a redeploy. Read the model ID from configuration. AWS AppConfig with Lambda and API Gateway supports this pattern.
2. Survive disruptions. A circuit breaker stops calls to a failing model. Graceful degradation returns a smaller model or a cached answer.
3. Customize with the least effort. Fine-tune on SageMaker AI when prompts and RAG fail. Use parameter-efficient methods such as LoRA.
4. Retire safely. Register each model version in SageMaker Model Registry. Plan rollback before you deploy.



Official AWS image. Source: [AWS AppConfig User Guide, What is AppConfig]. Use for the configuration-driven model switch pattern.

Exam trap. "No code changes" or "no redeploy" points to the configuration-driven pattern only.

1.2 Customization ladder

Rung	What changes	Use when
Prompt engineering	Instructions only	Try this first.
RAG	Private or current facts at query time	Facts change, and citations are needed.
Supervised fine-tuning	Model parameters, from labeled examples	Stable behavior that prompts cannot hold.
Distillation	A smaller student model learns from a larger teacher	A narrow task at high volume.

Fine-tuning does not fix facts that change. Use RAG for those facts.

1.2 Model lifecycle

Every release should be a version you can roll back. A model registry groups versions and tracks approval status. CloudTrail records Amazon Bedrock API calls, including InvokeModel and Converse, for the audit trail.

Exam trap. A model or prompt that changes in place has no safe rollback point.

1.2 Three ways to call a model

API	Use it for	Limit
Converse	Chat and tool use through one request format	Text, images, documents, and video for supported models. Not an embeddings API.
InvokeModel	Provider-specific bodies and embeddings	You build each provider's request body.
ConverseStream	Tokens as they arrive	The client must read a stream.

Streaming does not shorten total generation time. It shortens the time to the first visible token.

1.2 Structured output

A schema is stronger than a request for JSON. Structured output support depends on the model and the API. Check the API reference for the exact field. Check the model card before you rely on it.

Exam trap. A prompt that asks for JSON does not guarantee valid JSON. Add the schema.

1.3 Data validation and processing in four stages

1. Validate. AWS Glue Data Quality rules, Lambda checks, and CloudWatch metrics.
2. Process. Amazon Bedrock Data Automation for documents, images, audio, and video. SageMaker Processing for batch jobs. Amazon Transcribe for speech.
3. Format. JSON for Bedrock API requests. Conversation format for dialog apps.
4. Enhance. Amazon Comprehend extracts entities and detects PII. Lambda normalizes records.

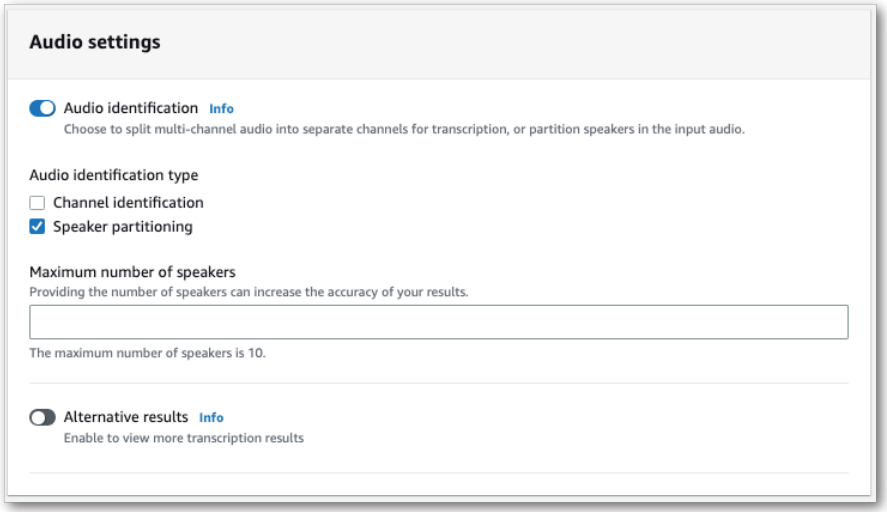
Need	Service
Extract fields from documents, images, video, and audio	Amazon Bedrock Data Automation
Rule-based data quality checks	AWS Glue Data Quality, with rules written in DQDL
Speech to text with speaker labels	Amazon Transcribe
Find and redact PII in text	Amazon Comprehend PII detection

Exam trap. Guardrails block, mask, or filter content. They do not validate data quality or extract invoice fields.

1.3 Services that look right and are wrong

Status notice. Amazon Kendra is no longer open to new customers. The Kendra documentation names Amazon Bedrock Knowledge Bases as the alternative.

Service	Looks right because	Why it is wrong here
Amazon Kendra	Enterprise search over documents	Not open to new customers. AWS points new work to Bedrock Knowledge Bases.
Amazon ElastiCache	Fast in-memory reads	Not a durable store of record. Durable only with durability enabled on node-based Valkey clusters.
Bedrock Guardrails	Reads document text	Checks content against safety policies. Does not extract data.
Amazon Translate	Changes the language of text	Translates text. Use Comprehend to detect PII.



Official AWS image. Source: Speaker diarization in Amazon Transcribe. Use for speaker labels.

1.3 Chunking by document type

Strategy	Use when	Note
Fixed size	FAQs and short, self-contained text	Simple to run.
Hierarchical	Manuals, contracts, long documents with headings	Small child chunks match. Parent chunks give context.
Semantic	Quality matters more than cost	Splits on meaning. Costs more to process.
Custom with Lambda	Tables, equations, figures with captions	Boundaries follow the document structure.

Exam trap. A table split at a chunk boundary loses its values. Each part can look correct alone.

1.3 Retrieval symptoms

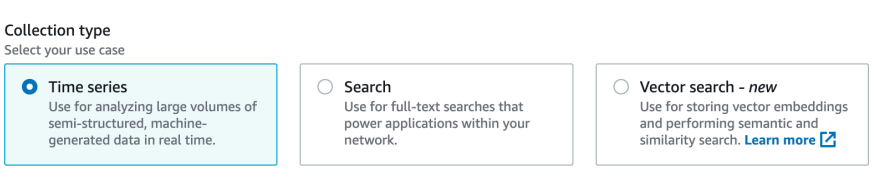
Chunks look relevant and answers are wrong. The failure is in retrieval, not generation. Changing the generator model does not fix broken retrieval. After a re-sync, check the embedding model version and the chunking configuration. To change chunking on a Knowledge Base data source, add a new data source with the new setting, then sync it.

1.4 Vector stores

Option	Fits
Bedrock Knowledge Bases	Private documents with citations. Managed RAG. The default answer.
OpenSearch Serverless	Managed vector search that scales with usage. Combines filters and vectors.
Aurora with pgvector	Vector search in a relational database the team already runs.
S3 Vectors	Large corpus, infrequent queries, no cluster to run.

DynamoDB	Metadata and embeddings in one table design.
Neptune	GraphRAG. Traverses explicit relationships between entities.

S3 Vectors fits large, rare searches. An always-on cluster costs money even when few queries arrive. For steady, interactive search with high query volume, an always-on store fits better. The cheapest store is wrong when the latency need is strict.



Official AWS image. Source: Amazon OpenSearch Serverless overview. Use for collection type choice.

Exam trap. Neptune is a distractor for plain document search. ElastiCache is a trap when the question needs a durable store.

1.4 Embeddings

Embeddings turn meaning into vectors. Amazon Titan Text Embeddings v2 outputs 1024 (default), 512, or 256 dimensions. Cohere Embed uses an `input_type` value. Use `search_query` for questions and `search_document` for passages. Call embedding models with InvokeModel, not Converse.

Keyword search misses paraphrases such as "reset my password" and "change my credentials". Vector search matches them. Smaller dimensions cost less and may lower quality. Test both sizes on a golden set and pick the one that passes.

1.5 RAG end to end

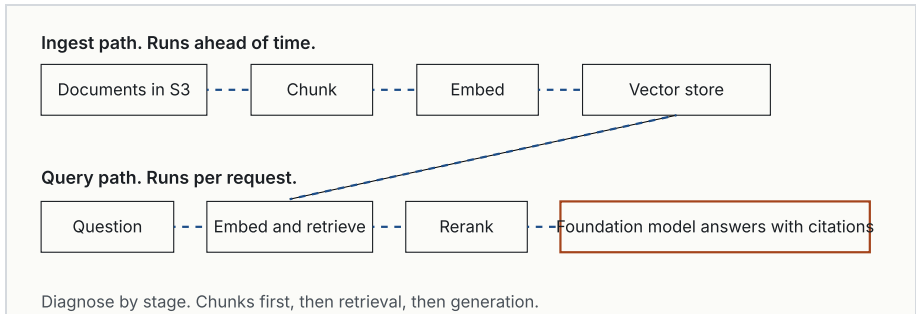
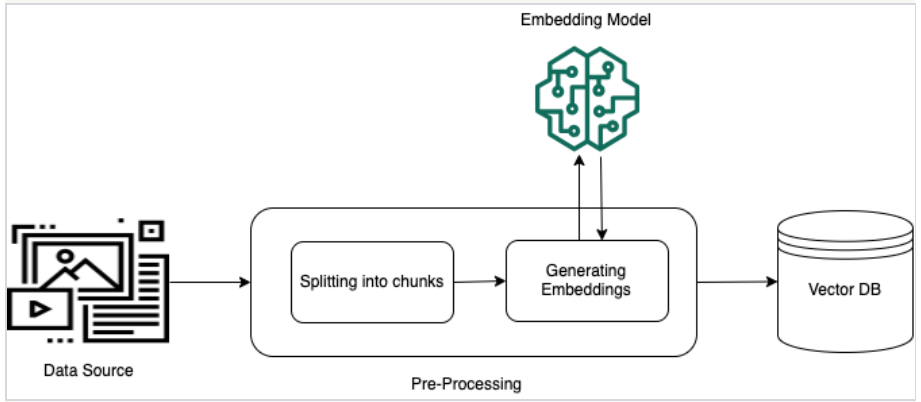
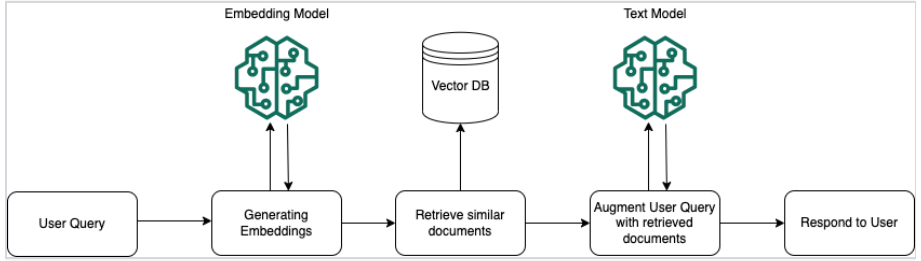


Diagram: two paths. The ingest path prepares the store. The query path answers each question. Built in SVG for this treatise.

Each stage is a design decision. A failure in one stage looks like a failure in the answer. Diagnose the stage that failed.



Official AWS image. Source: How Knowledge Bases work.



Official AWS image. Source: How Knowledge Bases work.

1.5 Retrieval fixes

Problem	Fix
Keyword-heavy queries drift	Hybrid search that combines semantic and keyword signals
Top results are related but not right	Reranker that scores candidates for relevance before generation
Vocabulary gap, such as refund and return	Query expansion
Multi-hop question	Query decomposition into sub-questions, then merge the results

Hybrid search finds exact codes and citation strings with keywords. It finds related meaning with vectors. A reranker orders candidates. It does not add chunks that retrieval missed.

Exam trap. Bigger models do not fix broken retrieval or broken chunking.

1.5 Knowledge Bases or hand-built RAG

Choose Bedrock Knowledge Bases for private documents with citations and the least custom effort. A sync and one API call cover the flow. Choose hand-built RAG only when a stated requirement, such as a custom retrieval algorithm, needs it.

Web crawler connectors cannot be created on customer-managed knowledge bases after September 30, 2026. Use Amazon Bedrock Managed Knowledge Base for web sources. This date comes from the Domain 1 fact-check. Confirm it on the Knowledge Base documentation before exam day.

Exam trap. A hand-built pipeline is a distractor when the scenario asks for least effort.

1.5 Citations and grounding

RetrieveAndGenerate returns a generated answer with source citations. Citations give attribution for audits. Contextual grounding checks return a score for each answer. Auditors can sample those scores.

1.6 Prompt governance

1. Instruct. Store role definitions and variables in Amazon Bedrock Prompt Management. Enforce responsible AI rules with Bedrock Guardrails.
2. Govern. Keep parameterized templates under version control. Use approval workflows before publication. Track use with CloudTrail.
3. Test. Use Lambda to check expected output. Use Step Functions to run edge cases.
4. Improve. Give structured inputs, a stated output format, and feedback loops.

1.6 Prompt Flows or Step Functions

Need	Use
A fixed prompt chain with branches, built in the console	Amazon Bedrock Prompt Flows
A durable workflow with waits, retries, and human approval	AWS Step Functions with callback task tokens
A workflow that waits about two days for a person	Step Functions Standard. Runs for up to one year.

Exam trap. For a multi-day approval, use Step Functions Standard. A Lambda that waits for a person hits the 15-minute limit.

1.6 Tenant and region isolation

Isolation must be structural. Use one knowledge base for each tenant. Use IAM to scope each role to its own knowledge base. Use metadata filters for region or product line. The model receives only what retrieval returns. A prompt that says "ignore other tenants" is not isolation.

Tools for agents: MCP

The Model Context Protocol (MCP) sets a standard way for agents to find and call tools. Remote MCP servers use Streamable HTTP. MCP is the tool connection standard. It is not a model choice or a guardrail.

Domain 1 gaps

Not in source: official sample questions for Domain 1. Named multimodal embedding models such as Titan Multimodal. Amazon SageMaker JumpStart, Neo, and Unified Studio.

Row map for Domain 1

Each table row points to its visual unit. Rows marked Text only have no unit yet.

Table row or topic	Unit or status	
Approach ladder: four rungs	#u21	
Knowledge or behavior table	#u21	
Five model axes	#u22	
Routing mechanisms	#u3 and #u20	
Customization ladder	#u20 and #u21	
Converse, InvokeModel, ConverseStream	#u1	
Structured output with a schema	#u55	Text only. No unit yet.
Model lifecycle and rollback	#u56	Text only. No unit yet.
Data tools table	#u23	
Services that look right and are wrong	#u24 for vector stores. #u63 for Kendra, Translate, ElastiCache.	
Chunking by document type	#u6	
Chunk shape for tables and structure	#u6	
Retrieval symptoms and re-sync	#u57	Text only. No unit yet.
Retrieval fixes	#u7	
Vector stores	#u24	
S3 Vectors for large, infrequent search	#u24	
Embedding dimensions	#u25	
Embeddings and input type	#u25	
Prompt governance	#u26	
Prompt Flows or Step Functions	#u27	
Knowledge Bases or hand-built RAG	#u58	
Knowledge Base data sources	#u59	
Citations and grounding	#u60	
Tenant and region isolation	#u61	
MCP tools	#u62	

PART 3 · DOMAIN 2 · WEIGHT 26 PERCENT (REPOSITORY README)

Implementation and integration

VISUAL UNITS FOR THIS DOMAIN

VISUAL UNIT 28

Agent loop with limits

First principle. An agent repeats reason, act, and observe until a stop condition. Each repetition costs time and money. Limits belong in the workflow or the tool layer, not only in the prompt.



- Stop conditions bound the number of steps.
- Timeouts in Lambda and Step Functions bound time.
- IAM bounds what each tool can touch.

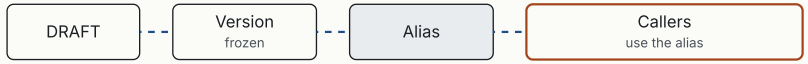
Exam tie. Domain 2 task 2.1.

Exam trap. A prompt that says stop is not a control.

VISUAL UNIT 29

Agent versions: DRAFT, prepare, alias

First principle. Changes go to DRAFT first. Callers use an alias that points to a version. Move the alias to put a change in production.



- Prepare the agent after you edit it.
- Create a version. Point the alias to it.
- Test the alias, not only DRAFT.

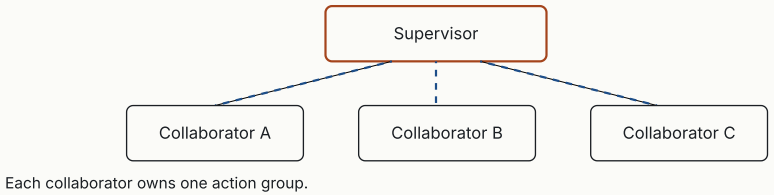
Exam tie. Domain 2 task 2.1 and Domain 5 task 5.2.

Exam trap. A console test on DRAFT does not prove the alias works.

VISUAL UNIT 30

Multi-agent collaboration: supervisor and collaborators

First principle. A supervisor reads the request and routes each part to a collaborator. Each collaborator has its own tools. The supervisor does not call the tools itself.



- The supervisor classifies intent in natural language.
- Each collaborator calls its own action group.
- A DRAFT collaborator cannot take delegations.

Exam tie. Domain 2 task 2.1.

Exam trap. Delegation is not an action group.

VISUAL UNIT 31

Deployment: match the purchase to the traffic

First principle. Model capacity is bought in shapes. On-demand suits spiky traffic. Provisioned Throughput suits a steady baseline. Batch suits offline work. Use a SageMaker endpoint for a model you own.



- Real-time endpoints for steady traffic.
- Serverless endpoints for intermittent traffic with cold starts.
- Asynchronous endpoints for jobs up to one hour.

Exam tie. Domain 2 task 2.2.

Exam trap. A real-time endpoint for a 45-minute job is the wrong choice.

VISUAL UNIT 32

Streaming and the gateway timeout

First principle. Streaming sends tokens as they appear. It lowers the wait for the first visible token. It does not extend a REST timeout of 29 seconds by default.



- Use WebSockets to hold a long stream.
- Use an asynchronous job for a very long call.
- Lambda timeout does not change the gateway limit.

Exam tie. Domain 2 tasks 2.3 and 2.4.

Exam trap. Raising the Lambda timeout does not fix a gateway timeout.

VISUAL UNIT 33

ApplyGuardrail on any model

First principle. A guardrail is a policy check. ApplyGuardrail runs the same check on text from any model. It does not call the model. Input and output checks are separate.



- Send content to ApplyGuardrail.
- Read the allow or block result.
- Use it for models outside Bedrock.

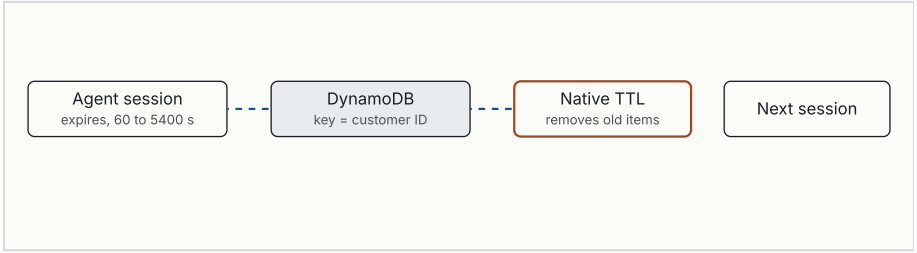
Exam tie. Domain 2 task 2.3 and Domain 3 task 3.1.

Exam trap. A non-Bedrock model does not need Converse to use ApplyGuardrail.

VISUAL UNIT 49

Session memory versus customer memory

First principle. An agent session expires. Customer memory must outlive the session. Store the memory in a table with a key and a time to live. The key is the customer or session ID.



- DynamoDB gives key lookup in single-digit milliseconds.
- Native TTL removes old records.
- The agent session TTL is between 60 and 5400 seconds.

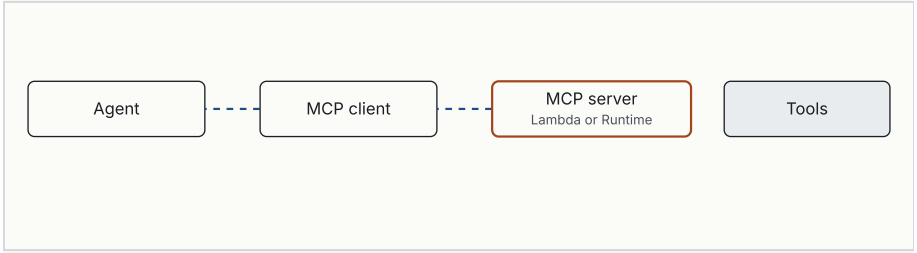
Exam tie. Domain 2 task 2.1.

Exam trap. The agent session is not the memory store. It expires.

VISUAL UNIT 50

MCP hosting and open-source frameworks

First principle. MCP is a standard way for an agent to find and call tools. A light, stateless tool can run on Lambda. A remote MCP server on AgentCore Runtime must use Streamable HTTP. Strands Agents and AWS Agent Squad are open-source frameworks named in the exam guide.



- Light, bursty tools: Lambda.
- Remote MCP server on AgentCore Runtime: Streamable HTTP.
- Frameworks give code-level control. Managed agents need less code.

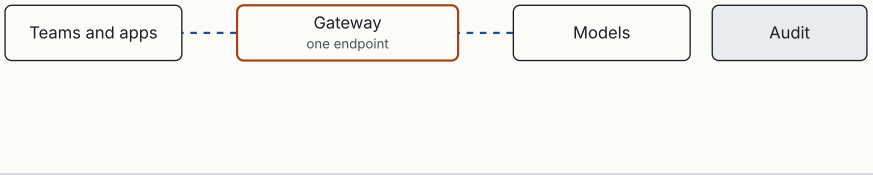
Exam tie. Domain 2 tasks 2.1 and 2.2.

Exam trap. A remote MCP server on AgentCore Runtime does not use another transport here.

VISUAL UNIT 51

Enterprise gateway for agent and model traffic

First principle. One gateway in front of many models and agents gives one place for identity, policy, logs, and cost tags. AgentCore Gateway routes agent traffic through one endpoint. Identity Center gives workforce users federated access.



- One endpoint for teams and apps.
- Logs and audit come from the gateway.
- Guardrail identifier and IAM policy apply on each call.

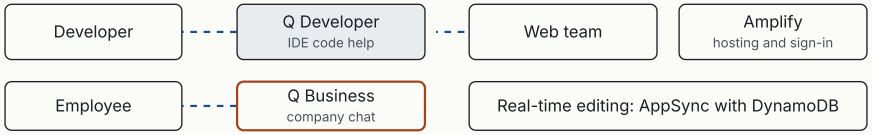
Exam tie. Domain 2 task 2.3 and Domain 3 task 3.3.

Exam trap. A gateway does not replace the guardrail. Enforce the guardrail at IAM as well.

VISUAL UNIT 52

Application tools: pick by user and task

First principle. The tool follows the user. A developer needs code help. An employee needs company chat. A web team needs hosting and sign-in. Match the tool to the person and the task.



- Q Developer: code help in the IDE.
- Q Business: company chat. No longer open to new customers.
- Amplify: managed web app with sign-in. AppSync: real-time editing.

Exam tie. Domain 2 task 2.5.

Exam trap. Q Business and Q Developer have similar names. The user decides the tool.

Domain 2 covers agents, deployment, enterprise integration, API patterns, and application tools. Its fact-check is still open. Read the official Bedrock Agents and Step Functions pages before exam day.

2.1 Anatomy of an agent

Status notice. Amazon Bedrock Agents is no longer open to new customers. The Bedrock Agents documentation names Amazon Bedrock AgentCore as the alternative. This section still teaches the agent model, because the exam guide tests it.

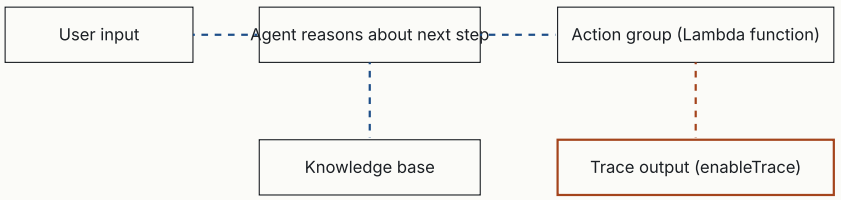
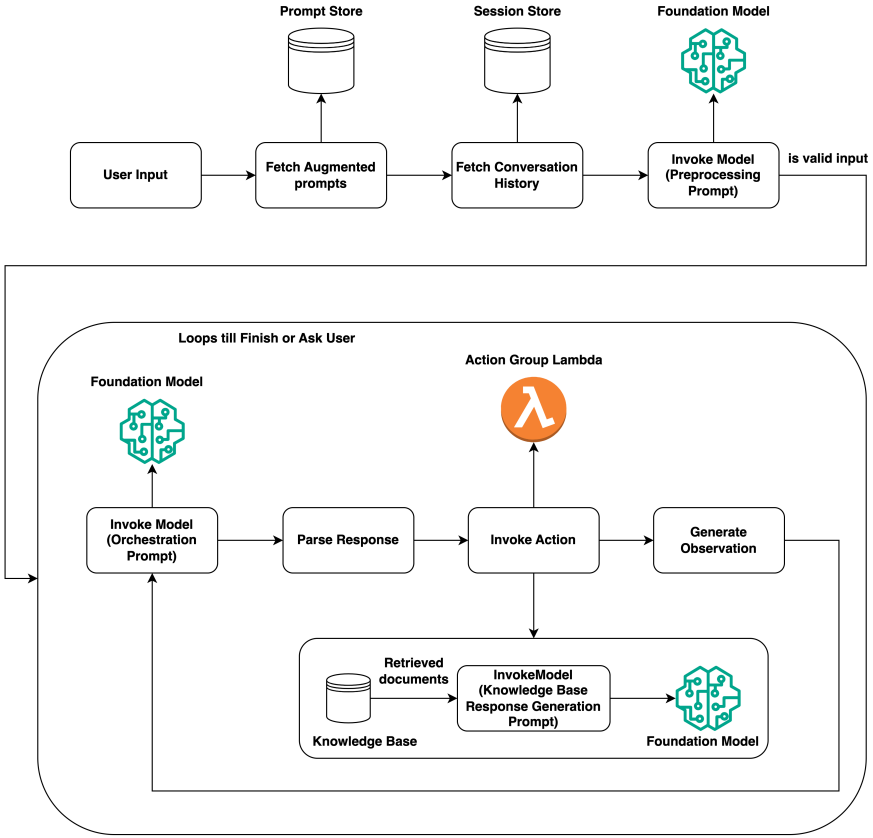
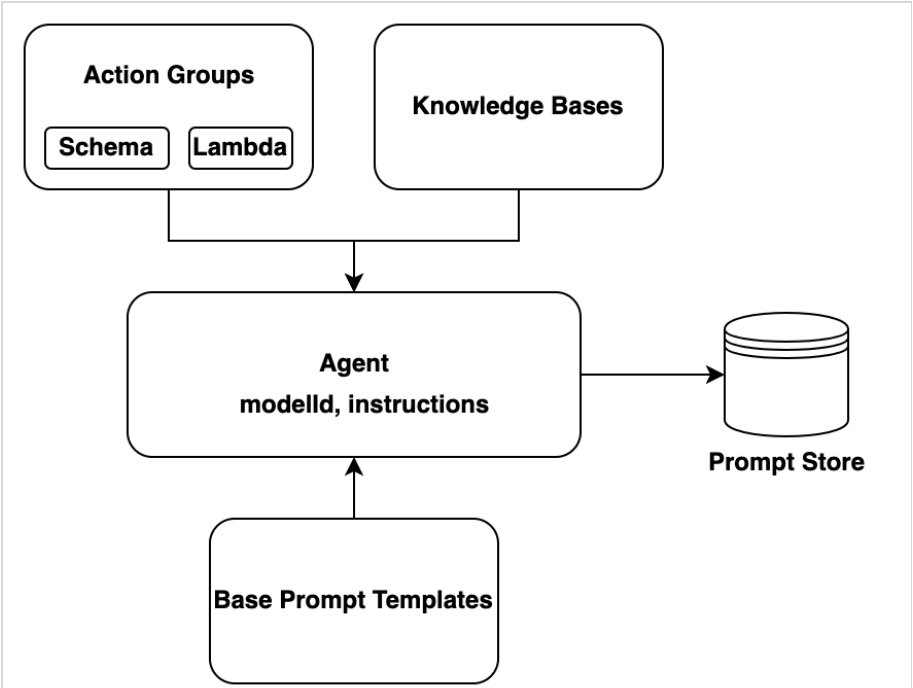


Diagram: the agent loop. The agent calls an action group or a knowledge base, and writes a trace record for each step. Built in SVG for this treatise.



Official AWS image. Source: Amazon Bedrock Agents, how agents work. Use for the runtime path.



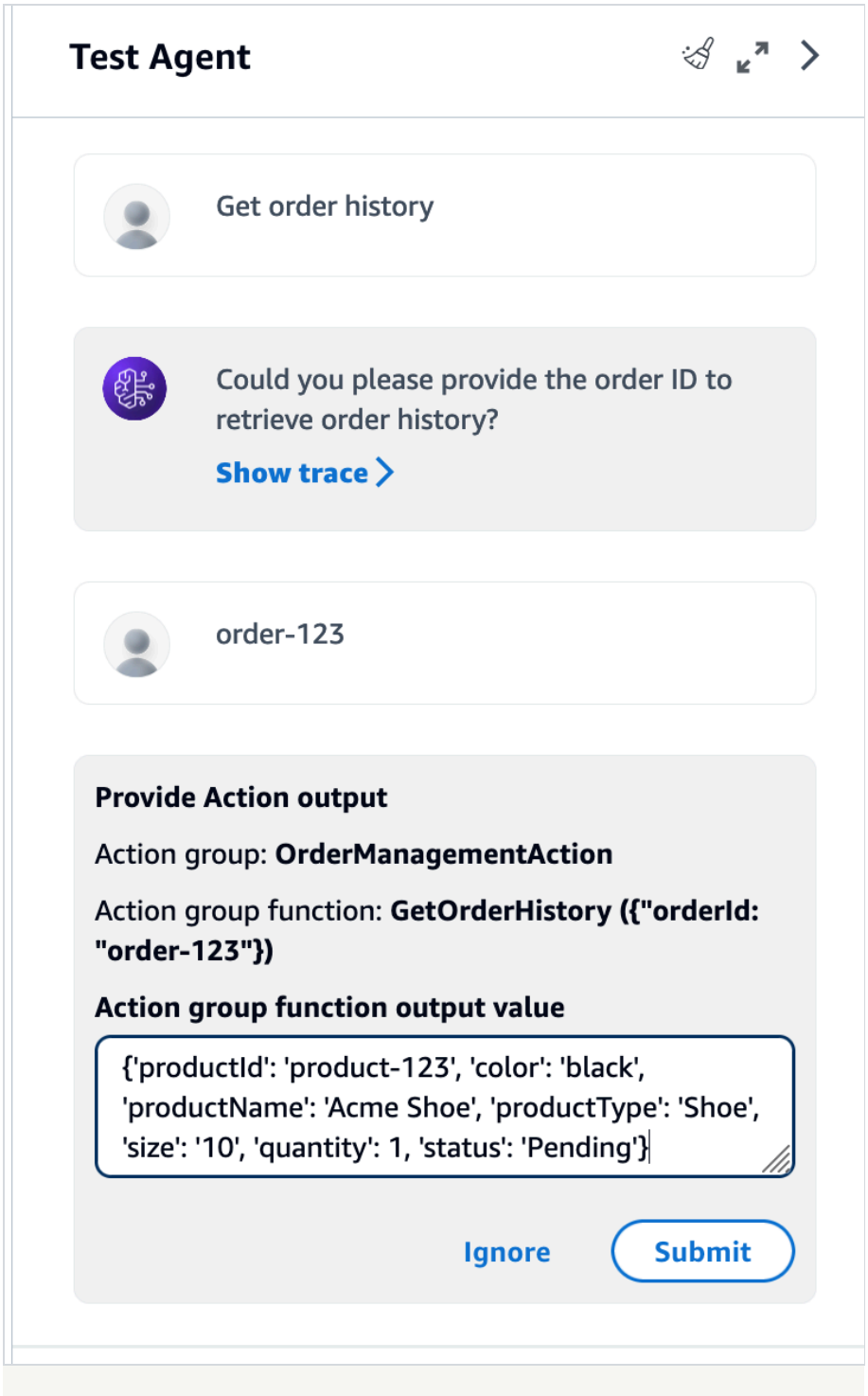
Official AWS image. Source: Amazon Bedrock Agents, how agents work. Use for the build-time path.

- `invoke_agent` takes `agentId`, `agentAliasId`, `sessionId`, and `inputText`. It returns a stream.
- The `sessionId` is the memory key for one conversation.
- Set `enableTrace` to get a record for each step.

Exam trap. After a change, prepare the agent and move the alias. Otherwise callers run the old version.

2.1 DRAFT, versions, and aliases

Configuration changes go to DRAFT. Callers reach a new version only after you create a version and point an alias to it. Console tests run against DRAFT. A passing console test does not prove alias traffic works. A new tool that works in the console but not in production usually means the alias still points to an older version.



Official AWS image. Source: Test an agent. Use to show the DRAFT test window.

2.1 Session memory and customer memory

The agent session expires. Customer memory must outlive it. Use DynamoDB for memory that lasts weeks. DynamoDB gives single-digit millisecond lookup by a key such as customer or session ID. It encrypts data at rest. Native TTL expires old records. The agent session TTL is set in seconds, between 60 and 5400.

2.1 Agent loop limits

Limit	Control
Stop	Stopping conditions bound the number of steps.
Time	Timeouts in Lambda and Step Functions.
Access	IAM policies set resource boundaries on every tool call.
Failure	Circuit breakers stop repeated failures to a tool.

2.1 Multi-agent collaboration

Bedrock multi-agent collaboration uses a supervisor agent and collaborator agents. Each collaborator calls its own tools through its own action group. Delegation is not an action group.

Exam trap. A DRAFT collaborator cannot take delegations. Prepare and alias it first.

2.1 Open-source frameworks and MCP hosting

Strands Agents and AWS Agent Squad are open-source frameworks named in the exam guide. Choose a framework when the team needs code-level control. Choose a managed agent when the console is enough.

Host light, stateless, bursty MCP tools on Lambda. Heavy, stateful servers need a different host. Their host is Not in source for this build. AgentCore Runtime requires Streamable HTTP for remote MCP servers.

2.2 Deployment patterns

Option	Fits
On-demand	Spiky or unknown traffic. Pay per token. No commitment.
Provisioned Throughput	A predictable baseline. Reserved model units. Hourly billing with a commitment term.
SageMaker AI endpoints	Custom or fine-tuned models. Real-time, serverless, or asynchronous.
Batch inference	Offline sets that can wait for hours. About half the on-demand price for select models.

For mixed traffic, use Provisioned Throughput for the baseline and on-demand for spikes.

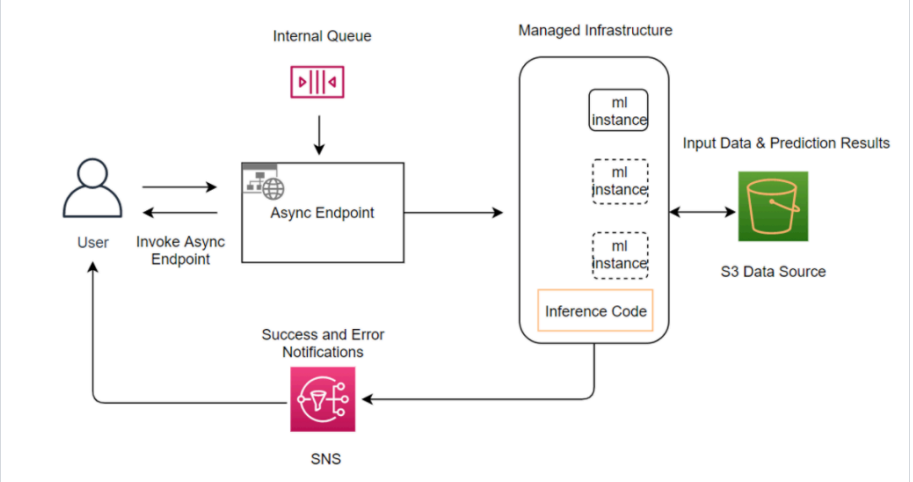
Exam trap. After buying Provisioned Throughput, calls to the base model ID still use on-demand capacity. Call the provisioned model ARN instead. Adding model units is the distractor.

2.2 SageMaker endpoints

Endpoint	Fit
Real-time	Steady traffic. Autoscaling. Blue-green and rolling deployments.
Serverless Inference	Intermittent traffic. Scales to zero. Trade-off: cold start on the first request.
Asynchronous	Long jobs, up to one hour. Queued. Results go to S3.



Official AWS image. Source: SageMaker AI serverless endpoints.

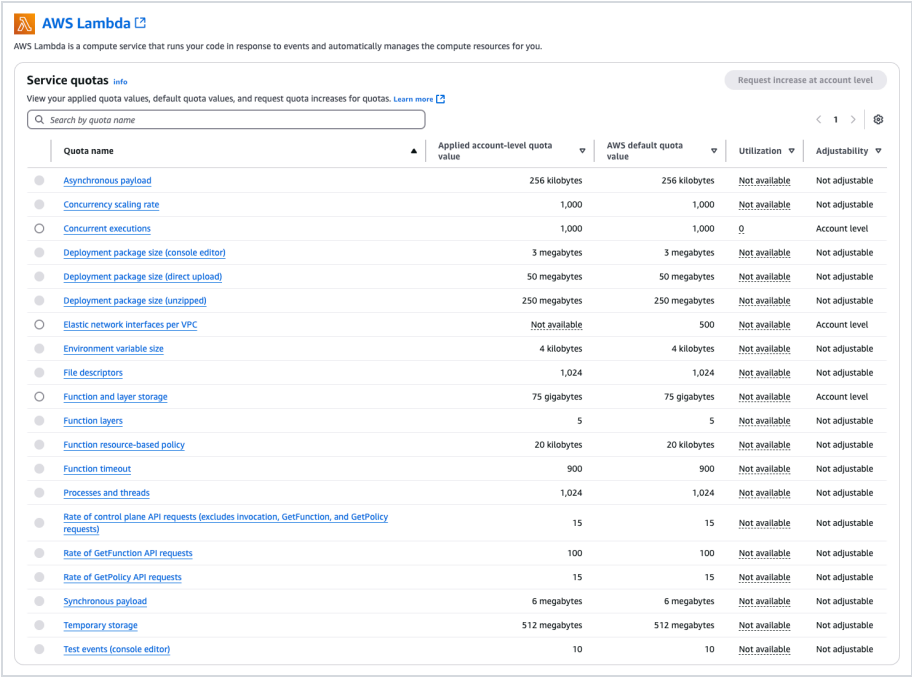


Official AWS image. Source: SageMaker AI asynchronous inference.

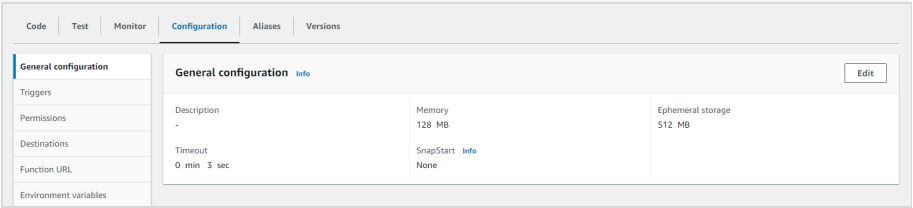
Exam trap. A real-time endpoint for a job that runs 45 minutes is the wrong pick. Use asynchronous inference.

2.2 Lambda limits and long work

A standard Lambda function runs for at most 15 minutes in a single invocation. Lambda Managed Instances functions invoked asynchronously allow up to 90 minutes. Do not assume the 90-minute case applies to a standard function. Put long flows in Step Functions and keep each Lambda step short. Waits in Lambda bill for the full duration. API Gateway REST integrations use a default integration timeout of 29 seconds. The quotas page fetched in this pass did not print that value, so confirm it on the API Gateway quotas page. Streaming does not change the timeout.

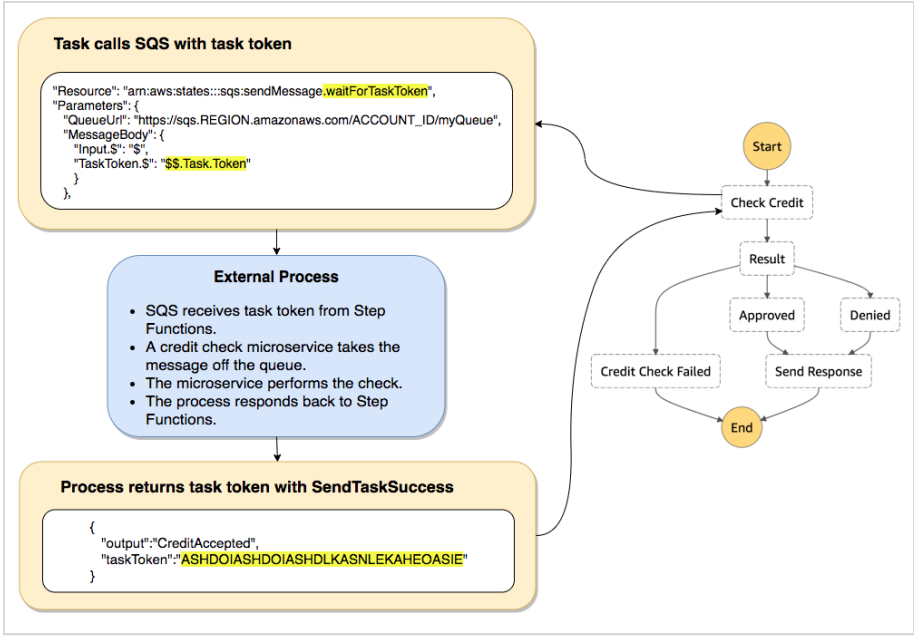


Official AWS image. Source: Lambda quotas and limits. Use for the Lambda execution model.

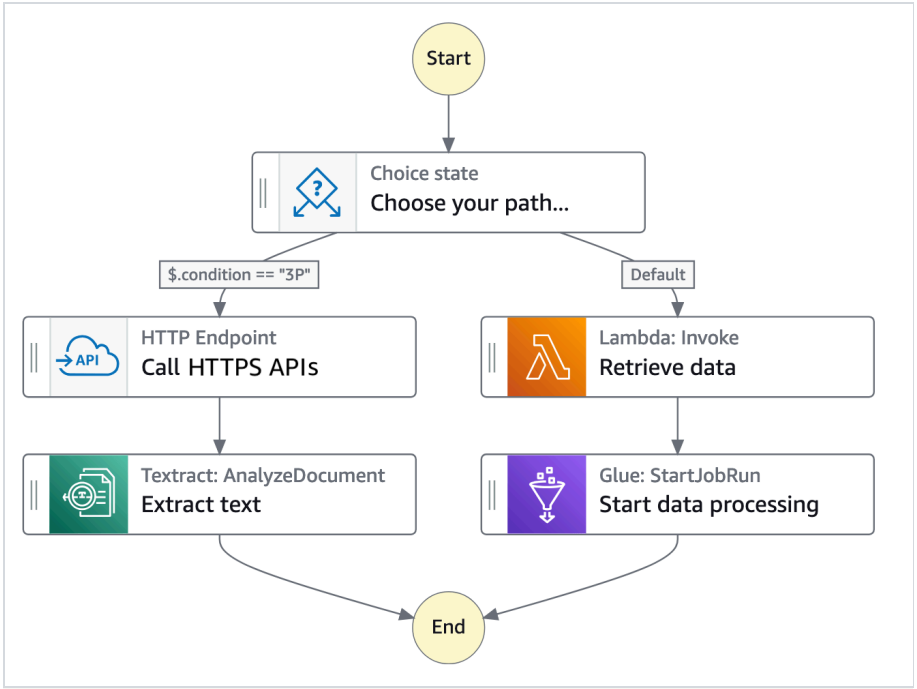


Official AWS image. Source: Lambda function timeout. Use for the timeout setting location.

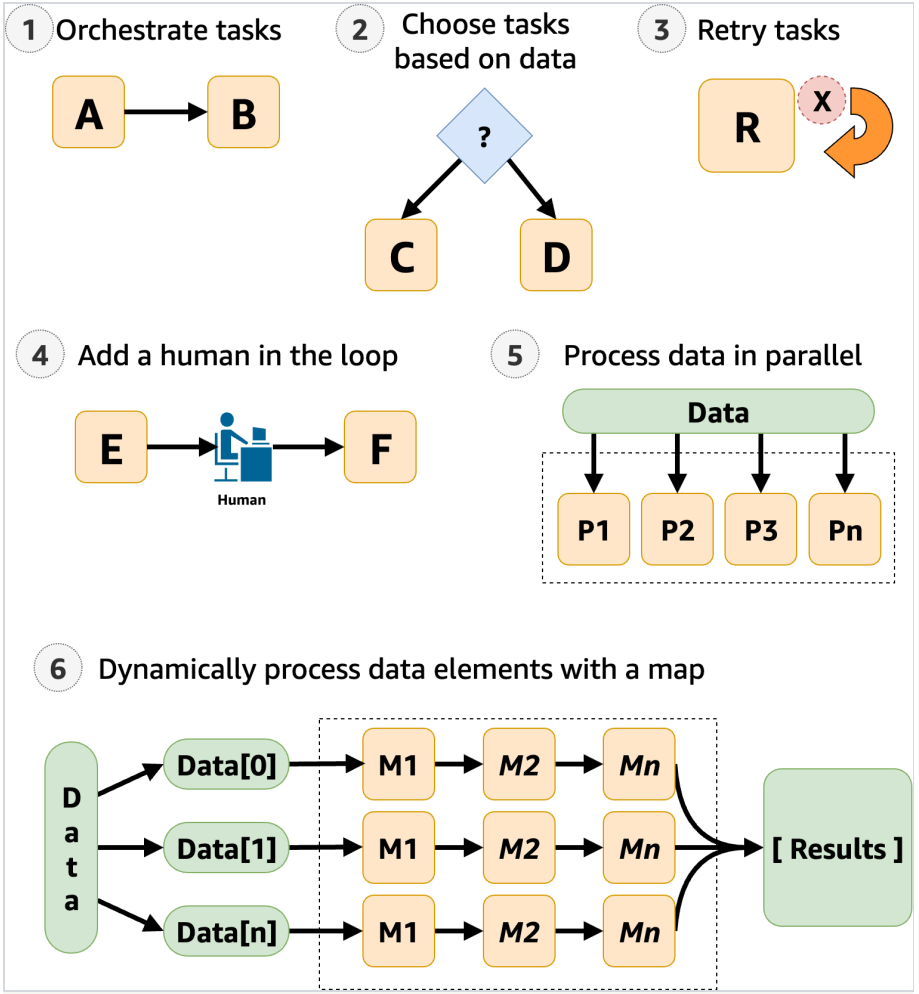
2.2 Human approval with a task token



Official AWS image. Source: Step Functions, connect to resources. Use for the callback pattern.



Official AWS image. Source: Wait for a callback with a task token. Use for the callback task token flow.



Official AWS image. Source: Wait for a callback with a task token.

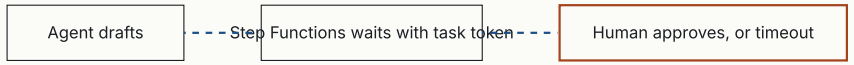


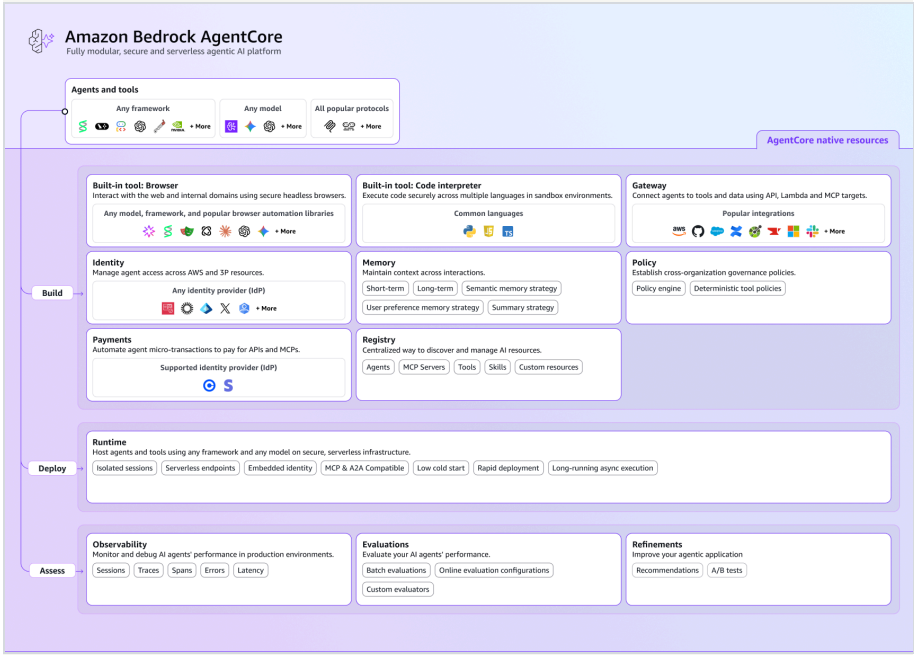
Diagram: a durable pause. Express workflows do not support the callback pattern. Built in SVG for this treatise.

The execution history gives the audit trail. Standard workflows keep execution history for up to 90 days. Use Standard for waits measured in

hours or days.

2.2 AgentCore components

AgentCore runs agents built with any framework behind one interface. Runtime hosts the agent code. Memory holds context. Gateway exposes tools as MCP tools. Identity controls who can call agents and tools. Observability records traces and metrics.



Official AWS image. Source: [What is Amazon Bedrock AgentCore](#).

2.3 Enterprise gateway and access

Put one controlled entry in front of the models. API Gateway, Lambda or EventBridge, and then Bedrock or SageMaker AI. One gateway enforces guardrail identifiers, IAM policy, uniform logs, and team cost tags. AWS Outposts keeps data on premises for rules that keep data on site. AgentCore Gateway routes agent traffic through one endpoint and includes observability and auditing.

2.3 Converse or InvokeModel

Task	API
Chat and tool use, one code path for many models	Converse
Embeddings	InvokeModel with an embedding model
Provider-specific body or parameter	InvokeModel. Claude requires `anthropic_version` and `max_tokens` in the body.

Exam trap. Converse is for text generation and chat. It is not the embeddings API.

2.3 ApplyGuardrail on any model

ApplyGuardrail applies a Bedrock safety policy to content from any model. It takes content and returns an allow or block result. It does not call the model. Input screening and output screening are separate checks.

2.4 Streaming to the browser

ConverseStream and InvokeModelWithResponseStream send tokens as they arrive. WebSocket APIs or server-sent events carry the stream to the browser. A REST integration still stops at the 29-second limit. Pick a path that can hold the stream.

Exam trap. A 90-second model call fails at the gateway. Raising the Lambda timeout does not fix it.

2.4 Call models reliably

1. Call. Use Bedrock APIs for synchronous calls. Use SQS for asynchronous work that can finish later.
2. Recover. Use exponential backoff with jitter. Set a maximum attempt count. Use a fallback model.
3. Observe. Trace calls with AWS X-Ray. Log token counts per call.
4. Route. Use static configuration, Step Functions for content-based routing, or metric-based routing.

Exam trap. A tight retry loop is never correct. Retries need jitter, a maximum attempt count, and a fallback.

2.5 Application tools

Need	Service
Coding help in the IDE, code changes, test generation	Amazon Q Developer
Chat over company data for employees	Amazon Q Business. No longer open to new customers. The documentation names Amazon Quick as the alternative.
Web app with managed hosting and sign-in	AWS Amplify
Real-time collaborative editing	AWS AppSync with DynamoDB
Extract data from documents, images, audio, and video	Amazon Bedrock Data Automation
No-code prompt chains with branches	Amazon Bedrock Prompt Flows

Exam trap. Q Business and Q Developer have similar names. One does company chat. The other writes code.

2.5 Prompt variants, versions, and caching

A variant compares two wordings side by side. A version is an immutable snapshot that you can roll back to. Prompt caching reuses a repeated prefix. The cached part must match exactly and sit before the changing content. Cache writes cost extra, so caching pays off only with repeat volume.

Exam trap. Prompt caching lowers the cost of a repeated prefix. It does not reserve throughput. Provisioned Throughput reserves capacity.

Domain 2 gaps

Not in source: official sample questions for Domain 2. AWS CodeDeploy, Kiro, and Amazon Managed Streaming for Apache Kafka. The fact-check for this domain has not closed.

Row map for Domain 2

Each table row points to its visual unit. Rows marked Text only have no unit yet.

Table row or topic	Unit or status
Agent parts and invoke_agent	#u28
DRAFT, versions, and aliases	#u29
Multi-agent collaboration	#u30
Agent loop limits	#u28
Open-source frameworks and MCP hosting	#u50
Session and customer memory	#u49
Deployment patterns	#u31
SageMaker endpoints	#u31
Lambda limits and long work	#u10
Human approval with a task token	#u10
AgentCore components	#u9
Enterprise gateway and access	#u51
Converse or InvokeModel	#u1
ApplyGuardrail on any model	#u33
Streaming and the gateway timeout	#u32

Call models reliably	#u12
Application tools	#u52
Prompt variants, versions, and caching	#u4 and #u26

PART 4 · DOMAIN 3 · WEIGHT 20 PERCENT (REPOSITORY README)

AI safety, security, and governance

VISUAL UNITS FOR THIS DOMAIN

VISUAL UNIT 34

Six guardrail policies, each for one failure

First principle. Each policy stops one kind of failure. Denied topics stop a subject. Word filters stop exact strings. Content filters stop harm and prompt attacks. Sensitive information filters stop PII. Grounding checks stop unsupported claims. Automated Reasoning checks stop claims that break formal rules.



- Denied topics: subject in plain language.
- Word filters: exact strings.
- Content and prompt attack: harm and injection on input.

Exam tie. Domain 3 task 3.1.

Exam trap. A word list cannot stop every phrasing of a subject.

VISUAL UNIT 35

Detect and redact: privacy at each stage

First principle. PII can appear in documents at rest, in input text, and in model output. Each place needs its own control.



- Macie finds sensitive data in S3 at rest.
- Comprehend detects PII in single text documents up to 100 KB.
- Guardrails redact or block PII at the model call.

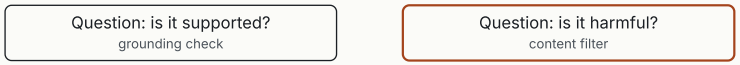
Exam tie. Domain 3 task 3.2.

Exam trap. Macie does not watch live model traffic.

VISUAL UNIT 36

Grounding versus safety

First principle. A grounding check asks whether the answer is supported by the source. A safety check asks whether the text is harmful. These are different questions with different controls.



Two questions. Two controls. Neither one removes every error.

- Contextual grounding: supported and relevant.
- Content filters: harmful text.
- Automated Reasoning: formal policy rules.

Exam tie. Domain 3 task 3.1 and Domain 5 task 5.2.

Exam trap. Grounding checks test support, not safety.

VISUAL UNIT 37

Defense in depth: four layers

First principle. Each layer uses a different control. A gap in one layer is covered by another. Put the cheapest check first and the strictest check at the model boundary.



- Pre-filter with PII detection and an abuse classifier.
- Guardrails on input and output.
- Lambda format checks. API Gateway throttling.

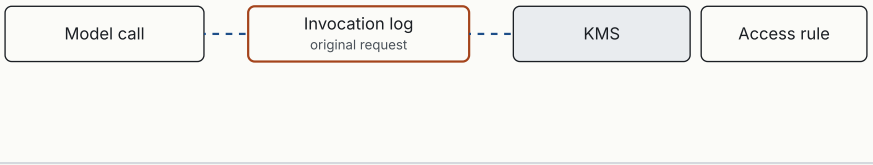
Exam tie. Domain 3 task 3.1.

Exam trap. No single control prevents all injection.

VISUAL UNIT 38

Logs that hold prompts

First principle. Invocation logs hold the request and response. Masking in the output does not mask the log. Protect the log store with KMS and access rules.



- Enable invocation logging for content.
- Encrypt with KMS. Restrict access.
- Object Lock gives write-once retention in S3.

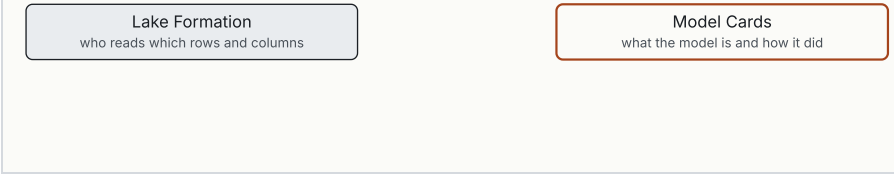
Exam tie. Domain 3 task 3.2 and Domain 4 task 4.3.

Exam trap. Invocation logs keep the original request even when output is masked.

VISUAL UNIT 64

Lake Formation and Model Cards: two different questions

First principle. Lake Formation controls who reads which rows and columns in a data lake. Model Cards document what a model is for and how it performed. One answers who reads data. The other answers what the model is.



- Column and row access: Lake Formation.
- Intended use and risk rating: Model Cards.
- Neither replaces CloudTrail.

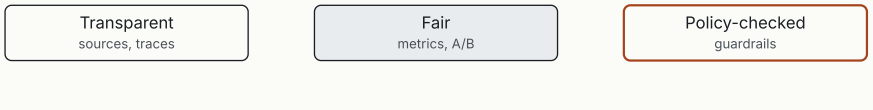
Exam tie. Domain 3 tasks 3.2 and 3.3.

Exam trap. Model Cards describe the model. Lake Formation controls data access. Check the Model Card JSON schema for the fields.

VISUAL UNIT 65

Responsible AI: transparent, fair, policy-checked

First principle. Responsible AI is three checks on one system. This is design guidance, not an AWS rule. Transparency shows sources and reasoning. Fairness is measured against defined metrics. Policy checks apply the rules the business requires.



- Transparent: sources, traces, citations on each claim.
- Fair: metrics defined before launch, A/B tests, judge evaluations.
- Policy-checked: guardrails and compliance checks in Lambda.

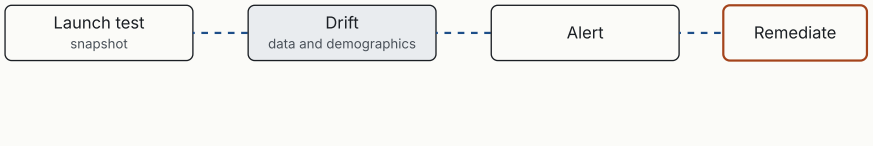
Exam tie. Domain 3 task 3.4.

Exam trap. A model card alone does not explain one decision. Add the citations and the trace.

VISUAL UNIT 66

Continuous fairness: a snapshot is not a guarantee

First principle. A fairness test before launch shows the state at one time. Data drift and demographic drift change behavior later. Design guidance: monitor fairness over time, and let a remediation workflow act on each alert.



- Define the fairness metrics before the build.
- Monitor the metrics after launch.
- Act on each alert with a remediation workflow.

Exam tie. Domain 3 tasks 3.3 and 3.4, and Domain 4 task 4.3.

Exam trap. A one-time bias test before launch does not prove fairness later.

Domain 3 covers guardrails, data privacy, compliance evidence, and responsible AI. Four failure modes need four controls. No single control covers all four.

Failure	Control	Layer
Confident wrong answer	Contextual grounding checks	Output
Prompt injection and jailbreak	Prompt attack filter	Input
PII leak	Sensitive information filters	Input and output

Harmful content	Content filters and denied topics	Input and output

3.1 One guardrail, six policies

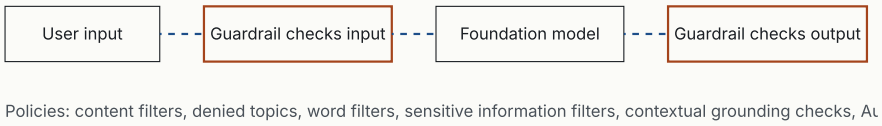
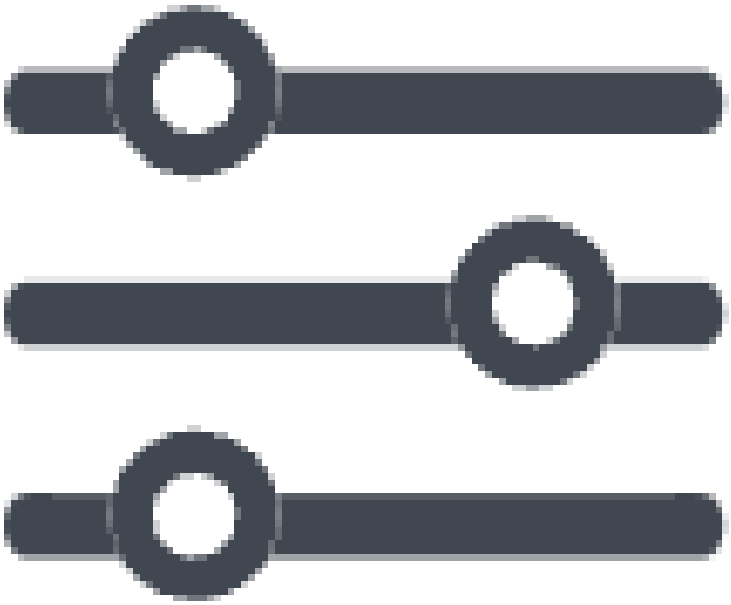


Diagram: the guardrail checks input and output. Built in SVG for this treatise.

Policy	Blocks or checks
Content filters	Harmful categories. The prompt attack category applies to input.
Denied topics	A whole subject, described in natural language.
Word filters	Exact words or phrases.
Sensitive information filters	PII entities. Redact or block.
Contextual grounding checks	Whether the answer is supported by the source and relevant to the question.
Automated Reasoning checks	Whether the response agrees with formal policy rules.

A draft is editable. A version is a frozen snapshot. Production points at a version. Input and output checks are separate switches.



Official AWS image. Source: Test a Knowledge Base. Use to show a configuration panel. Not a guardrail screen.

3.1 Choose the right filter

- Use a denied topic for a whole subject. A word list cannot stop every phrasing of a subject.
- Use a word filter for exact strings. It misses rephrased requests.
- Use the prompt attack filter for jailbreak and injection patterns. A denied topic does not detect prompt attacks.

Exam trap. Turn off the prompt attack filter and jailbreaks reach the model. Attack detection runs on input.

3.1 Grounding is not safety

Contextual grounding checks test support from the source. They do not test harm. Use content filters for harmful text. Grounding reduces hallucination. It does not remove it.

Exam trap. Answers that say a control "prevents all prompt injection" or "eliminates hallucinations" are wrong. No control is absolute.

3.1 Defense in depth

1. Pre-filter with Amazon Comprehend PII detection and an abuse classifier.
2. Guardrails on input and output.
3. Lambda format checks, allow-list checks, and business rules.
4. API Gateway throttling and usage plans.

Each layer fails sometimes. Layers use different controls, so a gap in one layer can be covered by another.

3.1 Detect mode

Detect mode runs the classifier and reports the verdict. It does not refuse the message. Use detect mode to measure a new rule before you block with it.

Exam trap. Detect mode is not a block. A blocked message needs the Block action.

3.2 Privacy: detect, then redact or block

Service	Where	Job
Amazon Macie	Data at rest in Amazon S3	Discovers sensitive data in S3 objects.
Amazon Comprehend	Single text documents	Real-time PII detection for single documents up to 100 KB.
Bedrock Guardrails	Model input and output	Sensitive information filters block or mask PII at the model call.

Exam trap. Macie does not watch live model traffic. Guardrails does not scan a whole bucket.

3.2 Private network access

VPC interface endpoints (AWS PrivateLink) keep API traffic on the AWS network. Create one endpoint for each Bedrock plane the workload uses. The bedrock-runtime plane runs invocations. The bedrock-agent and bedrock-agent-runtime planes serve agents.

Exam trap. One runtime endpoint does not cover agent calls. Check each plane.

3.2 Logs that hold prompts

Model invocation logs can hold prompts and completions. In a clinical setting, those logs can hold protected health information. Encrypt the logs with AWS KMS and restrict access. In a strict compliance setting, disable logging.

S3 Object Lock gives write-once-read-many retention. S3 Lifecycle expiration deletes objects at the end of their lifetime. Invocation logs keep the original, unmodified request, even when a guardrail masks PII in the output.

Exam trap. Masked output does not mean the log is masked. Protect the log store.

3.2 Data security plane

Plane	Control
Network	VPC interface endpoints keep traffic off the public internet.
Access	IAM least privilege. Lake Formation for column and row access to lake tables.
Encryption	AWS KMS for data at rest. TLS for data in transit.
Monitoring	CloudTrail records API calls. CloudWatch Logs holds invocation logs.

Encryption answers the storage risk. It does not answer a question about training use. Check the data terms and the retention settings for that question.

3.2 Cross-Region guardrail inference

Cross-Region guardrail inference routes requests to another Region in the same geography. Input prompts and outputs can move to that Region. A strict single-Region rule conflicts with that routing. Check whether the guardrail allows cross-Region inference before you choose it for a strict residency rule.

3.3 Data lake access and model documentation

Need	Control
One team sees a salary column and another does not	Lake Formation column-level access
A team sees only its own rows	Lake Formation row-level access
Versioned record of intended use, limits, and evaluation results	SageMaker Model Cards linked to the model package

Exam trap. Model Cards describe the model. Lake Formation controls who reads the data.

3.3 Audit and lineage

CloudTrail shows who called which API, when, and from where. It does not show row content. Lineage shows which documents support an answer. Combine catalog metadata with citations on each output. No single feature gives full lineage.

3.3 Governance mapping

Need	Service
Model documentation for audit	SageMaker Model Cards
Data lineage and source registry	AWS Glue Data Catalog
Audit of API calls	AWS CloudTrail
Decision and runtime logs	Amazon CloudWatch Logs
Runtime safety	Bedrock Guardrails

Map each regulation to one control, one proof, and one review cadence.

3.3 Organization-wide controls

Service control policies (SCPs) apply rules across accounts. A condition on the `bedrock:GuardrailIdentifier` key can require a guardrail on each request. Application code can be changed or skipped. The IAM layer still applies.

Exam trap. A guardrail that only the application code calls can be skipped. Enforce it at IAM.

3.4 Responsible AI

Principle	How to show it
Transparent	Show sources and reasoning. Use agent traces. Cite sources on each claim.
Fair	Define fairness metrics before launch. A/B test prompt variants. Evaluate at scale with an LLM-as-a-judge.
Policy-checked	Apply guardrails from policy requirements. Run compliance checks in Lambda.

Fairness is continuous. A launch test is a snapshot. Data drift and demographic drift change model behavior after launch. A one-time bias test before launch does not prove fairness later.

Exam trap. A model card alone does not explain one loan decision. Add the citations and the trace.

Domain 3 gaps

Not in source: official sample questions for Domain 3. The Lake Formation permission model details. IAM Access Analyzer. AWS WAF rule configuration.

Row map for Domain 3

Each table row points to its visual unit. Rows marked Text only have no unit yet.

Table row or topic	Unit or status
Four failures and their controls	Text plus #u37
Denied topics, word filters, prompt attacks	#u34
Six guardrail policies	#u34
Content filters table	#u34
Privacy controls	#u35
PII across the lifecycle	#u35
Grounding and reasoning checks	#u36
Defense in depth	#u37
Logs that hold prompts	#u38
Data security plane	#u17 and #u16
Audit and lineage	#u16
Organization-wide controls with SCPs	#u18
Governance mapping	#u16 and #u18
Cross-Region guardrail scope	#u3
Lake Formation and Model Cards	#u64
Responsible AI table	#u65
Continuous fairness	#u66

PART 5 · DOMAIN 4 · WEIGHT 12 PERCENT (REPOSITORY README)

Operational efficiency and optimization

VISUAL UNITS FOR THIS DOMAIN

VISUAL UNIT 39

Token cost: input plus output

First principle. Each token costs money. Input and output have different prices. Cost equals input tokens times the input price, plus output tokens times the output price, per million tokens.



Example: 4,000 input and 1,000 output tokens at 3 and 15 USD per million = 0.027 USD.

- Output usually costs more per token.
- Cap output length to cap cost.
- Prices in examples are illustrative.

Exam tie. Domain 4 task 4.1.

Exam trap. More capacity does not fix a cost that comes from long prompts.

VISUAL UNIT 40

Cost levers match the waste

First principle. Each waste has one lever. Long repeated prefixes need caching. Simple requests need smaller models. Off-hours work needs batch. Steady load needs reserved capacity.



- Prefix repeats: prompt caching.
- Simple queries: tiered routing.
- Offline work: batch. Steady load: Provisioned Throughput.

Exam tie. Domain 4 task 4.1.

Exam trap. Buying more capacity does not fix long prompts.

VISUAL UNIT 41

Latency: find the slow step first

First principle. End-to-end latency adds the steps. Measure each step before a fix. A slow retrieval step needs a retrieval fix, not a bigger model.



- Streaming improves perceived speed.
- Index tuning and query preprocessing fix search.
- Precompute predictable answers.

Exam tie. Domain 4 task 4.2.

Exam trap. A bigger model does not fix a slow search step.

VISUAL UNIT 42

Throttling: backoff, jitter, limit, fallback

First principle. Quotas cap tokens per minute and per day. A throttled call should wait, retry with jitter, and then fall back. Count tokens before the call to avoid overflow.



- Watch InvocationThrottles in CloudWatch.
- Retry with exponential backoff and jitter.
- Use CountTokens to check the context limit first.

Exam tie. Domain 4 task 4.2 and Domain 2 task 2.4.

Exam trap. Retries that add load make an overload worse.

VISUAL UNIT 43

Monitoring: which signal answers which question

First principle. Metrics show rate and cost. Traces show the slow step. Logs show the content. Anomaly alarms catch spikes that a fixed threshold misses.



- CloudWatch metrics and anomaly detection for spend.
- X-Ray for the slow step.
- Invocation logs for exact content.

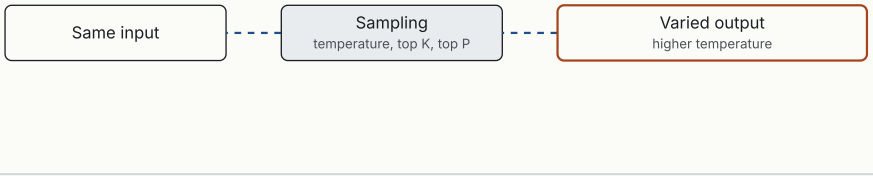
Exam tie. Domain 4 task 4.3.

Exam trap. A fixed threshold alarm misses slow growth.

VISUAL UNIT 53

Sampling and variance: randomness is a setting

First principle. A model picks each token from a distribution. Temperature sets how wide the choice is. Top K limits the candidate count. Top P limits the probability mass. Lower values give more repeatable output.



- Temperature: lower gives more repeatable output.
- Top K and Top P narrow the candidates.
- Sampling does not change the price per token.

Exam tie. Domain 4 task 4.2 and Domain 5 task 5.2.

Exam trap. Buying capacity does not fix different answers to the same input.

VISUAL UNIT 54

Three similar concepts: profile, router, reserved capacity

First principle. Three different objects look alike. A cross-Region profile spreads requests across Regions. A router picks between two models in one family. Provisioned Throughput reserves model units. Each one answers a different question.



- Profile: throughput and availability.
- Router: cost, by predicted quality.
- Reserved capacity: steady traffic, called with the provisioned ARN.

Exam tie. Domain 4 tasks 4.1 and 4.2.

Exam trap. Code that calls the base model ID does not use reserved capacity.

VISUAL UNIT 67

Retrieval cost: fewer, better chunks

First principle. Retrieval adds tokens to every call. Each chunk you send costs input tokens. Send fewer chunks, but rank them first so the best ones remain.



- Lower the number of results the retriever returns.
- Rerank, then cut to the top results.
- Keep chunks sized so one answer stays whole.

Exam tie. Domain 4 tasks 4.1 and 4.2.

Exam trap. Sending more chunks raises token cost.

Domain 4 covers cost, latency, capacity, throttling, and monitoring. Match each observed fact to one lever.

4.1 Token economics

You pay for each input token and each output token. The formula uses the price per million tokens.

Cost = (input tokens / 1,000,000 × input price) + (output tokens / 1,000,000 × output price)

Illustrative prices: 3 USD per million input tokens and 15 USD per million output tokens. Not current AWS

Example: 4,000 input and 1,000 output tokens = 0.012 + 0.015 = 0.027 USD.

Diagram: the cost formula with illustrative prices. Built in SVG for this treatise.

At these illustrative prices, output costs five times as much as input per token. Long replies raise cost. Set a maximum output length to cap the reply. Check current AWS prices before exam day.

4.1 Cost levers

Observed fact	Lever
Most requests are simple	Tiered routing to a smaller model
The same long prefix repeats	Prompt caching with a cache checkpoint
Steady baseline load with a latency target	Provisioned Throughput for the baseline
Short spikes above the baseline	On-demand inference
Work that can wait for hours	Batch inference
Many paraphrases of the same question	Semantic cache

Exam trap. Buying more capacity does not fix a cost that comes from long prompts.

4.1 Prompt caching

A cache checkpoint marks the static prefix of a prompt as cacheable. Keep the variable part at the end. A changing user message in the prefix breaks the cache. The default time to live is 5 minutes. The prefix must meet a minimum token count. The fact-check found minimums from 512 to 4,096 tokens, depending on the model.



Official AWS image. Source: Prompt caching in Amazon Bedrock. Use to show where caching is enabled.



Official AWS image. Source: Prompt caching in Amazon Bedrock. Use to show cache read and write metrics.

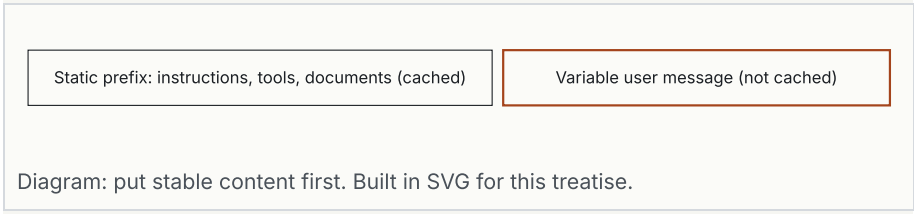


Diagram: put stable content first. Built in SVG for this treatise.

4.1 Semantic cache

A semantic cache matches paraphrases of the same question by embedding similarity. It returns the stored answer before the model runs. Prompt caching is different. It reuses an exact prompt prefix at the model. A semantic cache needs repeated, similar questions to get a good hit rate. Highly personal questions rarely repeat.

4.1 Batch for work that can wait

Batch inference reads input from S3 and writes output to S3. The price is 50 percent lower than on-demand for select models. Set the job time limit with the timeout duration in hours. A user waiting for a chat reply needs on-demand, not batch.

4.1 Model right-sizing

Pattern	How it works
Cascade	Try the small model first. Escalate when the answer is uncertain or the query is complex.
Route by complexity	A classifier or router sends each query to a model. Bedrock intelligent prompt routing picks between two models in one family by predicted response quality and cost.
Fine-tune a small model	For a narrow, repeated task at high volume.

Exam trap. Sending every request to the largest model wastes cost.

4.1 Capacity concepts that look alike

Concept	Purpose	Choose for
Cross-Region inference profile	Routes requests across Regions for throughput and availability	Availability and resilience
Intelligent prompt routing	Routes between two models in one family	Cost

Provisioned Throughput	Reserves model units	Steady, predictable traffic

Exam trap. Code that calls the base model ID does not use reserved capacity. Call the provisioned model ARN.

4.1 Quotas, retries, and pre-call checks

1. Quotas. Tokens per minute (TPM) and tokens per day (TPD) cap token use. Watch the InvocationThrottles metric in CloudWatch.
2. Retry. Use exponential backoff with jitter. Set a maximum number of attempts.
3. Fall back. Use a second model or a cached answer when retries end.
4. Check first. Count tokens before the call with CountTokens. Compare the count with the model limit.

Exam trap. Retries that add load can make an overload worse. Back off with jitter.

4.2 Latency: find the slow step

Latency is what the user feels. Throughput is what the system handles. Stream the response to improve perceived speed. Streaming does not raise throughput. For a slow retrieval step, use index optimization, query preprocessing, or hybrid search. A bigger model does not fix a slow search step.

Goal	Lever
Perceived speed	Stream the response as it generates
Speed for predictable questions	Precompute answers ahead of time
Volume	Batch inference for offline work. Provisioned Throughput for steady capacity.

4.2 Sampling and variance

Temperature sets how wide sampling is. Lower values give more repeatable output. Top K limits the candidate tokens. Top P limits the share of the probability distribution that is considered. Sampling settings do not change the price per token. Buying capacity does not fix different answers to the same input.

Not in source: effect of a fixed seed on output. Effect of a versioned prompt on output variance. Effect of sampling on token counts.

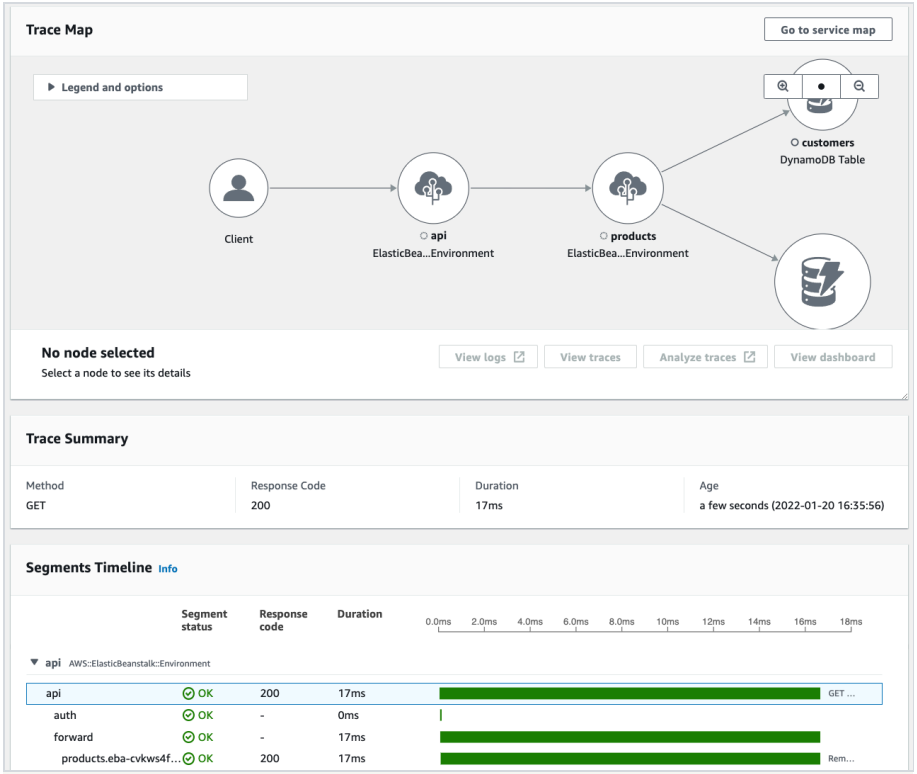
4.3 Monitoring signals

Signal	Tool and use
Token and invocation metrics	CloudWatch metrics: InputTokenCount, OutputTokenCount, Invocations
Spend spikes	CloudWatch anomaly detection and alarms. Cost Anomaly Detection for spend.
Exact requests and responses	Model invocation logs. Search with CloudWatch Logs Insights.
Slow or failing step	AWS X-Ray traces requests and downstream calls.

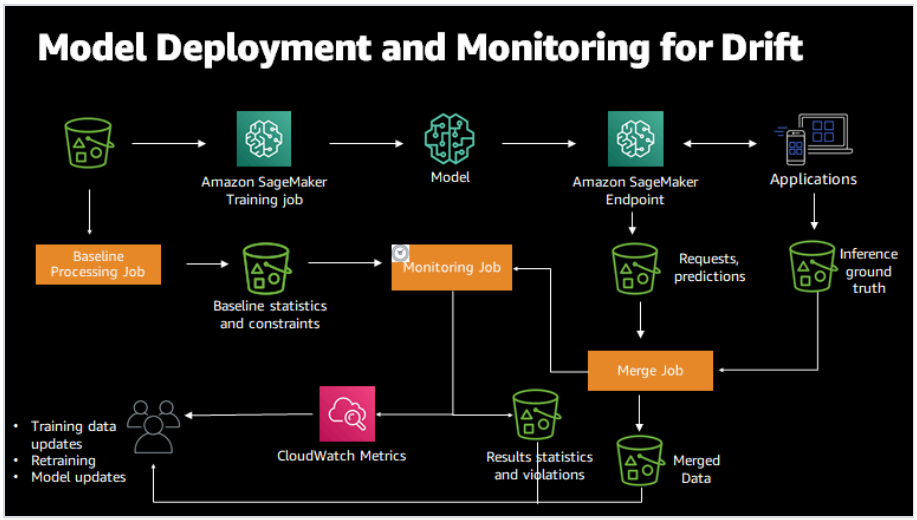
Pattern: find the slow step with X-Ray, then audit the exact prompts in the invocation logs.



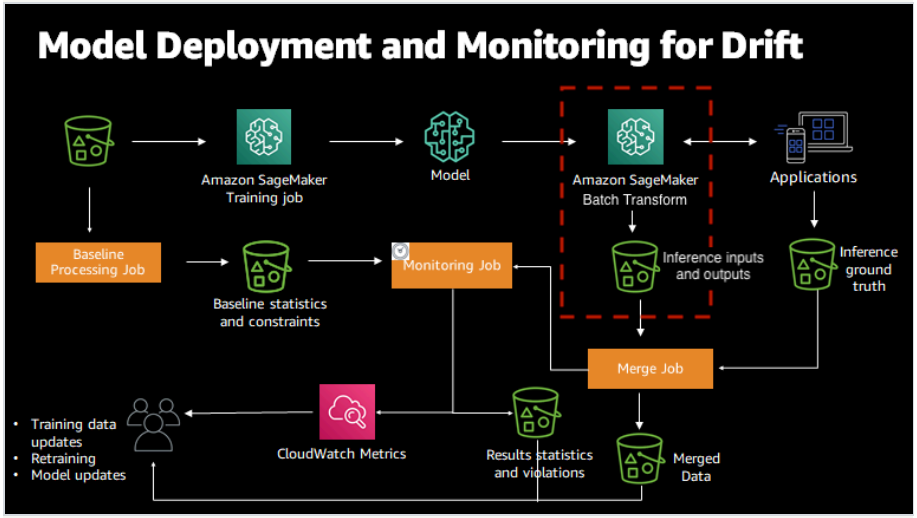
Official AWS image. Source: [AWS X-Ray concepts](). Use for the service map view.



Official AWS image. Source: [AWS X-Ray concepts](). Use for trace segments and timing.



Official AWS image. Source: SageMaker AI Model Monitor. Use for drift monitoring. The page states that Model Monitor is no longer open to new customers. Check that status before you rely on it.



Official AWS image. Source: SageMaker AI Model Monitor.

Exam trap. Model Monitor watches statistical drift in structured features and predictions. It does not score free-text answer quality. Use a judge or a golden set for that.

Domain 4 gaps

Not in source: official sample questions for Domain 4. Amazon CloudWatch Synthetics. AWS Auto Scaling for GenAI workloads. AWS Cost Explorer.

Row map for Domain 4

Each table row points to its visual unit. Rows marked Text only have no unit yet.

Table row or topic	Unit or status
Token economics	#u39
Cost levers	#u40
Prompt caching	#u4
Semantic cache	#u5
Batch for work that can wait	#u2
Model right-sizing	#u20
Capacity concepts that look alike	#u54
Quotas, retries, and pre-call checks	#u42
Latency and the slow step	#u41
Sampling and variance	#u53
Monitoring signals	#u43
Tracing, drift, and forensics	#u15
Retrieval cost and retrieval tuning	#u67

PART 6 · DOMAIN 5 · WEIGHT 11 PERCENT (REPOSITORY README)

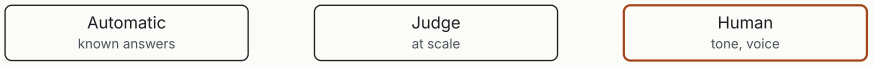
Testing, validation, and troubleshooting

VISUAL UNITS FOR THIS DOMAIN

VISUAL UNIT 44

Choose the cheapest evaluation method that measures the claim

First principle. A claim needs a method. Automatic metrics suit known answers. A judge suits quality at scale after calibration. Humans judge tone and voice.



- Automatic: accuracy, robustness, toxicity.
- Judge: correctness and faithfulness at scale.
- Human: brand voice and custom criteria.

Exam tie. Domain 5 task 5.1.

Exam trap. Automatic accuracy does not measure tone.

VISUAL UNIT 45

Split retriever from generator in RAG evaluation

First principle. A RAG answer has two parts. The retriever finds chunks. The generator writes the answer. A good answer can hide weak retrieval. Measure each part.



- Retriever: context relevance and context coverage.
- Generator: faithfulness to the chunks.
- A drop in retriever-only scores points at retrieval.

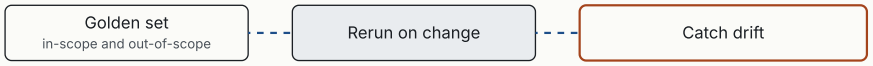
Exam tie. Domain 5 task 5.1.

Exam trap. Fine-tuning does not fix retrieval.

VISUAL UNIT 46

Golden set and hallucination tests

First principle. A golden set holds trusted prompts with known answers. A hallucination test needs questions whose answers are not in the documents. Without them, the set only confirms what the documents already support.



- Include out-of-scope questions.
- Rerun the set after every change.
- Compare answers across runs for inconsistency.

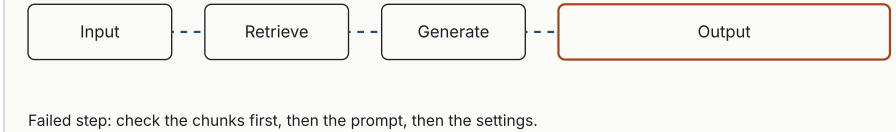
Exam tie. Domain 5 tasks 5.1 and 5.2.

Exam trap. A passing suite with new user failures means the golden set lacks those cases.

VISUAL UNIT 47

Troubleshooting: start at the failed step

First principle. A GenAI fault sits in one stage. Trace the request from input to output. Check the stage that changed last.



- Output cut off: check the maximum tokens.
- Answer ignores documents: check retrieved chunks.
- Throttling after buying capacity: check the model ARN.

Exam tie. Domain 5 task 5.2.

Exam trap. Temperature does not fix bad retrieval.

VISUAL UNIT 48

Release gates and canaries

First principle. A change to a prompt, model, chunking setting, or guardrail is a release. Each release passes the same gates. A failed gate blocks it. A canary limits the blast radius.



- Run the golden set on each change.
- Gate on the quality threshold.
- Canary first, then full traffic.

Exam tie. Domain 5 task 5.1 and Domain 2 task 2.3.

Exam trap. A re-sync is a change. It needs the same gate.

VISUAL UNIT 68

Adversarial and safety testing: test before users do

First principle. An attacker tries inputs you did not plan for. An automated test set sends known attack patterns to the system. Run the set again after each guardrail or prompt change.



- Build a test set of injection and jailbreak patterns.
- Run it after every guardrail or prompt change.
- Check the Bedrock evaluation documentation for the agent evaluation methods.

Exam tie. Domain 5 task 5.1 and Domain 3 task 3.1.

Exam trap. Red-team procedures are not in the source. Plan them separately.

Domain 5 covers evaluation methods, golden datasets, A/B tests, debugging, and release gates.

5.1 Evaluation matrix

Method	Measures	Use when
Automatic	Accuracy, robustness, toxicity. Accuracy needs ground-truth labels.	Objective, repeatable checks
LLM-as-a-judge	Correctness, faithfulness, harmfulness, style. Custom metrics.	Quality at scale. RAG and agent evaluation.

Human	Relevance, brand voice, custom criteria	Subjective judgment
Bring your own inference	The same metrics on your own responses	Non-Bedrock models or external RAG sources
Retrieve-only	Context relevance and context coverage	Isolate the retriever

Pick the cheapest method that measures the claim. Escalate only for what the cheap method cannot judge.

Exam trap. Automatic accuracy metrics do not measure tone, style, or voice. Use human review for brand voice.

5.1 Judges need calibration

1. Rate a fixed golden set with humans.
2. Run the same set through the judge.
3. Measure judge-human agreement. Low agreement means the judge is not valid.
4. Pin the judge model ID. A changed judge changes the scores.

Exam trap. An uncalibrated judge gives opinions, not measurements.

5.1 RAG evaluation: split retrieval from generation

Part	Metrics
Retriever	Context relevance and context coverage. Did the system get the correct articles?
Generator	Faithfulness. Are the claims supported by the retrieved chunks?

Citation precision and citation coverage are Not in source for this build. Context relevance and context coverage come from an AWS blog post, not a reference page.

Exam trap. A good final answer can hide weak retrieval. Measure both parts.

5.1 Golden datasets, regression, and canaries

Practice	What it does
Golden dataset	Trusted prompts with known good answers. Run them after every change.
Regression suite	Reruns the golden set on each change to catch drift.
Canary	Releases a new version to a small share of traffic first.
A/B test	Compares two variants on live or sampled traffic.

Exam trap. A re-sync or a prompt edit is a change. It needs the same release gate.



Diagram: a failed gate blocks the release. Built in SVG for this treatise.

5.1 Safety tests

Run automated adversarial test sets against prompt injection and jailbreak patterns. Run them again after every guardrail or prompt change. Agent evaluations measure task completion and tool usage. Step-by-step red-team procedures are Not in source.

5.2 Troubleshooting by symptom

Symptom	Likely cause	First check

Output cut off mid-sentence	Maximum output tokens too low, or context overflow	Maximum tokens and the token count against the limit
Answer ignores the documents	Retrieval or chunking	Retrieved chunks, before the model
Output changes between runs	Sampling settings or prompt drift	Temperature, Top P, prompt version
Throttling after buying capacity	Base model ID used	Provisioned model ARN in the call
Agent runs old behavior	Agent not prepared, or alias not moved	Prepare the agent. Check the alias version.
Works on Titan, fails on Claude	Wrong InvokeModel body	anthropic_version and max_tokens in the body
AccessDeniedException in one Region only	IAM permissions for that call	IAM policy and resource ARNs for that Region
Quality dropped after a re-sync	Changed chunking or embedding model	What the sync changed. Then the golden set.

Exam trap. Temperature does not fix bad retrieval. Start the debug at the step that failed.

5.2 Debugging tools

CloudWatch Logs shows what the model received. Logs Insights finds patterns by time, caller identity, or request metadata. CloudTrail shows who called which API and when. CloudTrail is for audit, not for debugging answers.

5.2 Grounding and hallucination

Tool	Detects
Golden dataset with out-of-scope questions	Hallucinations
Output comparison across runs	Inconsistent answers to the same input
Contextual grounding check and grounding score	Claims that the retrieved source does not support

A hallucination test needs out-of-scope questions. Without them, the set only confirms answers that the documents already support.

Domain 5 gaps

Not in source: official sample questions for Domain 5. Setup steps for Bedrock Model Evaluations jobs. Step-by-step red-team procedures. Bedrock Agents Classic is closed to new customers per the docs. The agent rows here apply to existing use.

Row map for Domain 5

Each table row points to its visual unit. Rows marked Text only have no unit yet.

Table row or topic	Unit or status
Evaluation matrix	#u44
Evaluation methods within a budget	#u44
Choose the evaluation method	#u44
Judge calibration	#u13
Subjective quality	#u44
RAG evaluation and RAG metrics	#u45

Golden datasets, regression, and canaries	#u46 and #u14
Release gates in the pipeline	#u48
Troubleshooting by symptom	#u47
Debugging tools and debug table	#u47
Hallucination control	#u46
Adversarial and safety testing	#u68

PART 7

Traps AWS reuses

Wrong answer	Correct idea
Fine-tune on documents that change	RAG with a Knowledge Base
Neptune for plain document search	A vector store. Neptune is for graph queries.
Converse for embeddings	InvokeModel with an embedding model
Batch for a user waiting on a reply	On-demand, real-time calls
Guardrails to extract invoice fields	Bedrock Data Automation
CloudTrail as a data trigger	S3 events and EventBridge
Prompt attack filter off	Keep it on for input
ElastiCache as the durable vector store	OpenSearch Serverless, Aurora with pgvector, or S3 Vectors
Absolute claims about injection or hallucination	No control is absolute
Agent runs without prepare	Prepare, then move the alias
Action group to link two agents	Built-in supervisor and collaborator pattern
More model units after throttling	Call the provisioned model ARN
Raise the Lambda timeout past 900 seconds	900 seconds is the Lambda maximum. Use asynchronous jobs or Step Functions.
Tight retry loop	Backoff, jitter, maximum attempts, fallback
Prompt caching for throughput	Provisioned Throughput for reserved capacity
Automatic accuracy metric for tone	LLM-as-a-judge and human review
Temperature to fix retrieval	Fix chunking and retrieval first
Hand-built RAG when the scenario asks for least effort	Bedrock Knowledge Bases

PART 8

Method for each question

1. Read the last sentence. Note what it asks for.
2. List each constraint in the stem. Cost, latency, compliance, and existing footprint.
3. Match each constraint to one service or pattern.
4. Eliminate each option that breaks one constraint.

5 For multiple response, count the answers the stem requires. Select all of them.

PART 9

Gaps, limits, and checks

Not in source for this build

- Official sample questions for any domain. Only third-party practice sets were found.
- Amazon SageMaker JumpStart, Neo, and Unified Studio.
- AWS CodeDeploy, Kiro, and Amazon Managed Streaming for Apache Kafka.
- Amazon CloudWatch Synthetics, Auto Scaling for GenAI workloads, and AWS Cost Explorer.
- Lake Formation permission model details, IAM Access Analyzer, and AWS WAF rules.
- Step-by-step red-team procedures and Model Evaluations job setup.

Open checks before exam day

1 Confirm the domain weights against the official AIP-C01 exam guide. The official certification page did not show them.

2 Confirm the 180-minute duration and the 300 USD fee on the official certification page.

3 Confirm the Domain 2 API Gateway integration timeout on the API Gateway quotas page.

4 Confirm the web crawler date and the Model Monitor status on the official pages.

5 Confirm whether Amazon Bedrock uses prompts for model training. The sources did not settle this.

Images

The images in this treatise come from AWS documentation pages. Each caption names its source page. They are for personal study. Icons and other decorative items from the pages were left out.